

The AI Builder's Handbook

Foundations, frameworks, and field notes
for building AI products.

Published by

LevelUp Labs



The AI Builder's Handbook

Foundations, frameworks, and field notes for building AI products.

Published by LevelUp Labs.

Copyright © 2026 LevelUp Labs. All rights reserved.

The content, frameworks, curriculum, and the selection and ordering of referenced resources are the intellectual property of LevelUp Labs. Third-party materials cited throughout remain the property of their respective publishers.

This guide is licensed under Creative Commons Attribution-NonCommercial 4.0 (CC BY-NC 4.0). You may share it freely with credit to LevelUp Labs. Commercial use or derivative works require prior written permission.

Contact levelup-labs.ai

Edition Public, 2026

Contents

Preface	1
What This Is	1
Scope	2
Three Ways to Read This	2
A Note on Evaluation	3
Where Our Resources Come From	3
One Last Thing	3
 Part 1: The Landscape	 4
 Chapter 1: The Language of Generative and Agentic AI	 5
1.1 The Raw Material	6
Generative AI	6
Foundation Model	6
Large Language Model (LLM)	7
Tokens	7
Context Window	7
Pretraining, Fine-Tuning, Post-Training	8
1.2 From Text to Thought	9
Chain-of-Thought	9
Reasoning Models	9
Multimodality	10
1.3 Talking to the Model	10
Prompting	10
System Prompt	11
Few-Shot and In-Context Learning	11
Context Engineering	11
1.4 Beyond Single Calls	12
Tool Use and Function Calling	12
Agent	12
Workflow vs Agent	13
Router and Classifier Patterns	13
Specialist and Sub-Agent	14
Multi-Agent System	14

1.5 Knowing What You Don't Know	15
Retrieval	15
Semantic Search and Embeddings	15
Vector Database	15
Memory	16
1.6 Keeping It Honest	16
Evaluation (Evals)	16
LLM-as-Judge	17
Hallucination	17
Grounding	18
Guardrails	18
1.7 Plumbing	18
Observability and Tracing	18
Model Context Protocol (MCP)	19
Agents SDK	19
The Map, Assembled	20
Chapter 2: What Enterprises Are Building	21
2.1 The Shift From Pilots to Production	21
2.2 Where the Value Is Actually Landing	22
2.3 The Patterns That Are Working	23
2.4 What Is Failing (and Why)	24
2.5 What This Means for Your Roadmap	25
Chapter 3: Models: How to Choose	26
3.1 There Is No Best Model	26
3.2 The Dimensions That Actually Matter	27
3.3 A Decision Framework That Works	28
3.4 Reasoning Models: A Different Kind of Tool	29
3.5 Open vs Closed	30
3.6 Your Evals Beat Any Benchmark	30
Part 2: Designing AI Products	31
Chapter 4: Problem-First Design, Revisited	32
4.1 Why Problem-First Matters More in AI	32
4.2 The Four Layers Framework	33
4.3 Six Traps We See Constantly	34
4.4 A Scoping Template You Can Use	35
4.5 Designing for Iteration	36
4.6 Problem-First Doesn't Mean Problem-Forever	37

Chapter 5: Prompting and Context Engineering	38
5.1 The Six Parts of a Working Prompt	38
5.2 Prompting for Reasoning Models Is Different	39
5.3 Context Engineering: The Bigger Picture	40
5.4 Few-Shot Examples: Your Highest-Leverage Tool	41
5.5 Output Format: The Interface Nobody Talks About	42
5.6 When to Stop Iterating on Prompts	42
 Part 3: The Evaluation Core	 43
Chapter 6: Why Evals Are the Real Work	44
6.1 The 80/20 Flip	44
6.2 What Evals Actually Buy You	45
6.3 The Two Questions Every Eval Answers	46
6.4 Evals Are Infrastructure, Not a Milestone	46
6.5 A Companion Course	47
6.6 How to Read the Next Three Chapters	47
 Chapter 7: Code-Based Evals	 49
7.1 What Code-Based Evals Can Check	49
7.2 When Code-Based Evals Are Enough	50
7.3 Building Your First Code-Based Eval	51
7.4 The Failure Modes Code-Based Evals Catch Best	52
7.5 The Limits	52
7.6 What You Need in Your First Eval Suite	53
 Chapter 8: LLM-as-Judge and Calibration	 54
8.1 When LLM-as-Judge Makes Sense	54
8.2 A Brand Voice Judge: A Worked Example	55
8.3 Writing a Judge Prompt That Works	55
8.4 Calibration: The Step Nobody Skips Twice	57
8.5 The Ways LLM Judges Fail	58
8.6 Judges as Part of a Full Eval Suite	58
8.7 When to Recalibrate	59
 Chapter 9: Guardrails: Input and Output	 60
9.1 What Guardrails Are	60
9.2 Input Guardrails	61
9.3 Output Guardrails	61
9.4 When Guardrails Replace Evals, and When They Supplement	62
9.5 The Strictness Tradeoff	63
9.6 Production Patterns That Work	64

9.7 The Right-Sized Guardrail Stack	64
Part 4: Building Agentic Systems	65
Chapter 10: From Single Calls to Agents	66
10.1 The Spectrum	66
10.2 Why the Spectrum Matters	67
10.3 A Simple Rule for Moving Up	68
10.4 When Agents Are Actually the Right Shape	69
10.5 The Hybrid Pattern That Usually Wins	69
10.6 Evaluating Systems as You Move Up	70
Chapter 11: Workflow and Router Patterns	71
11.1 The Five Workflow Patterns Worth Knowing	71
11.2 The Router Pattern in Depth	72
11.3 Designing a Router That Holds Up	73
11.4 When to Add Confidence and Human Review	74
11.5 Human-in-the-Loop Patterns	75
11.6 Workflow Composability	75
11.7 The Workflow Shape That Works for Document-Heavy Use Cases	76
Chapter 12: Tool Use and Actions	78
12.1 How Tool Use Actually Works	78
12.2 Designing a Good Tool	79
12.3 Read-Only Tools First	80
12.4 Permissioning: The Part Teams Skip	81
12.5 Error Handling When Tools Fail	81
12.6 A Reference Tool-Use Shape	82
12.7 When to Add a New Tool	83
Chapter 13: Retrieval	84
13.1 What Retrieval Actually Does	84
13.2 Semantic Search, Keyword Search, and Why You Usually Want Both	85
13.3 Chunking: The Decision Most Teams Get Wrong	86
13.4 Ranking and Reranking	87
13.5 Evaluating Retrieval	87
13.6 Retrieval Patterns Beyond the Basics	88
13.7 What to Do Before Shipping a Retrieval System	89
Chapter 14: Memory and Long-Running Agents	90
14.1 Short-Term and Long-Term Memory	90

14.2 When Memory Is Worth Adding	91
14.3 Short-Term Memory Patterns	92
14.4 Long-Term Memory: The Building Blocks	92
14.5 Memory Evaluations	93
14.6 Long-Running Agents	94
14.7 Memory Gotchas Worth Knowing	95
Chapter 15: Multi-Agent Systems	96
15.1 Before You Build a Multi-Agent System, Don't	96
15.2 When Multi-Agent Is Actually the Right Shape	97
15.3 The Main Multi-Agent Topologies	98
15.4 Communication Between Agents	98
15.5 Failure Modes You Should Expect	99
15.6 Evaluating Multi-Agent Systems	100
15.7 One Real Example: The Google AI Co-Scientist	100
15.8 The Honest Short Answer	101
 Part 5: Production and the Long Arc	 101
Chapter 16: Observability and Tracing	102
16.1 What Observability Looks Like for AI Systems	102
16.2 Why Traces Matter	103
16.3 What to Log (and What Not To)	104
16.4 The Metrics That Matter	104
16.5 From Traces to Evals and Improvements	105
16.6 The Minimum Observability Setup	106
16.7 The Part Nobody Talks About: Culture	106
 Chapter 17: Protocols and Extensibility	 108
17.1 Why Protocols Matter for Enterprise	108
17.2 Model Context Protocol (MCP)	109
17.3 Agent Protocols (A2A and Similar)	110
17.4 OpenAI Agents SDK	111
17.5 What to Do With All This	111
17.6 Making Your Own System Extensible	112
17.7 The Horizon for Protocols	113
 Chapter 18: Production Readiness Checklist	 114
18.1 The Core Principle	114
18.2 Pre-Launch Checklist	115
Problem and Scope	115
Evals	115

Guardrails	115
Observability	116
Safety	116
User Experience	116
Documentation	117
18.3 Post-Launch Checklist	117
Weekly	117
Monthly	117
Quarterly	118
Annually	118
18.4 The Signals You Are Drifting	119
18.5 A Word on Shipping Imperfect Systems	119

Part 6: Where to Go Next 120

Chapter 19: Role-Based Learning Tracks 121

19.1 Product Manager Track	121
19.2 UX Designer Track	122
19.3 Engineer Track	124
19.4 Shared Homework Across All Tracks	125
19.5 For Leaders Reading This	125

Chapter 20: The Horizon 127

20.1 Reasoning Keeps Getting Better and Cheaper	127
20.2 Long Context, Really	128
20.3 Multimodal by Default	128
20.4 Open Models Close More of the Gap	129
20.5 Autonomy, Carefully	129
20.6 Regulation and Governance	130
20.7 The Enterprise AI Platform	131
20.8 What Does Not Change	131
20.9 What to Do With This Chapter	132
20.10 Final Word	132

Master Resource Index 134

By Source	134
LevelUp Labs	134
Anthropic	135
OpenAI	135
Google and Google DeepMind	136
Meta	136
DeepLearning.AI	136

Observability Platforms	136
Retrieval and Vector Databases	137
Frameworks and Agent Tooling	137
Guardrail Tools	137
Protocols	138
Enterprise AI Reports	138
Regulation and Governance	138
Product and Design References	138
Benchmarks and Model Comparison	138
By Topic	139
Foundations and Vocabulary	139
Enterprise Landscape	139
Model Selection	139
Problem-First Design	139
Prompting and Context Engineering	139
Evaluation	140
Guardrails	140
Agents and Workflows	140
Tool Use	140
Retrieval	141
Memory	141
Multi-Agent	141
Observability	141
Protocols	142
Regulation and Governance	142
By Type	142
Free Courses	142
Provider Documentation Starting Points	142
Industry Reports (Annual or Ongoing)	143
Newsletters and Ongoing Sources	143

Preface

This is the AI Builder's Handbook. A self-paced learning path for people building AI products, from the vocabulary on day one through to production decisions, in twenty chapters.

Each chapter stands on its own and points at the primary source when you want to go deeper than a chapter can.

What This Is

A book-length guide, twenty chapters, organized as six parts. Each chapter pairs a clear walkthrough of a concept with a short list of hand-picked resources for going deeper. Read end to end, it takes you from the vocabulary of modern AI through the frameworks for designing, evaluating, building, and shipping AI systems, and ends with a view of what's coming next.

The pattern in each chapter is the same. We explain the concept in plain language, connect it to the workflow shape teams actually ship, and then point you at the primary source (a research paper, a provider's documentation, a course) for when you want the full depth.

No chapter is longer than it needs to be. No resource is on the list unless we trust it.

Scope

The examples are kept generic on purpose. You know your own domain better than we do. The patterns are what transfer across use cases.

The guide is opinionated. Where two approaches compete, we picked the one more common in production today. Where the field genuinely disagrees, we flagged the disagreement so you can form your own view.

One more thing worth saying upfront. Reading the guide does not substitute for building. Reading about evaluation does not make you good at evaluation. Reading about agent design does not teach you the taste that comes from building three agents and throwing away two. The guide is the scaffolding. The building is yours.

Three Ways to Read This

Linearly, start to finish. The arc is designed to build on itself. Part 1 establishes the landscape and the vocabulary. Part 2 teaches how to design a system. Part 3 goes deep on evaluation, which is the muscle we believe matters most. Part 4 covers the shapes of agentic systems. Part 5 is about shipping. Part 6 points you at the horizon.

By role. Chapter 19 has three curated reading orders: one for PMs, one for UX designers, one for engineers. Each track prioritizes the chapters most relevant to how your role ships AI.

By topic. If you are stuck on a specific problem (retrieval is hallucinating, your LLM judge does not agree with your humans, your agent is looping), skip to that chapter. The chapters work as standalone references after the first read.

A Note on Evaluation

We have threaded one thing through this guide more than any other: evaluation. Our belief, formed across thirty-plus enterprise implementations, is that teams who get serious about evals ship AI products that actually work, and teams who do not get stuck in a cycle of demos that never graduate.

Part 3 of this guide is our deepest treatment of the topic. It points in several places at [AI Evals for Everyone](#), a companion course that goes deeper on the mechanics than a chapter can.

Where Our Resources Come From

Primary sources first. When we cite a concept, we try to link to a paper, to the model provider who implemented it, or to the infra company that built around it. Secondary takes are tempting but they age poorly and they carry their author's agenda.

We also pull heavily from LevelUp Labs' own [awesome-generative-ai-guide](#) on GitHub. It is the most active curated resource in the space and it is free. The glossary images, the paper roundups, the free courses all live there.

DeepLearning.AI, the free learning platform from Andrew Ng's team, shows up often. It is the cleanest free technical education in this field.

One Last Thing

We spent weeks on this guide because we wanted it to be worth the hours you will spend with it. If you find things that are wrong, things that are unclear, or things you wish were covered, email us. The guide will evolve.

Keep building.

LevelUp Labs

Start with [Chapter 1: The Language of Generative and Agentic AI](#).

Chapter 1: The Language of Generative and Agentic AI

You just spent a day inside a field whose vocabulary doubles every six months. Terms you hear casually in one meeting carry completely different weight in another. “Agent” might mean a Python script that loops through tool calls, or an autonomous system that plans, acts, and recovers on its own. “Eval” might mean a unit test, a scorecard, or a full human-labeled benchmark.

This chapter is your ground truth. A shared map of the vocabulary, organized the way builders actually use it.

We have organized it the way you would actually build understanding: start with the raw material (what a model is and how it reads text), then move outward to how you talk to a model, then to what happens when you let a model act on the world, then to how you keep it honest and observable. Read it top to bottom the first time. Come back to it as a reference whenever a term starts to feel slippery.

A quick note before we start. The AI field is bad at vocabulary. Two reputable sources will define “agent” differently, and both will sound right. Wherever we have to pick a definition, we follow the usage closest to how the work actually gets done in production. We flag the places where other people use the term differently.

Every term is paired with at least one primary source. If the resource line looks interesting, treat that as your next step.

1.1 The Raw Material

Before anything else: the system you are building on top of is a probability machine trained to predict text. Every other concept in this guide extends from that one fact.

Generative AI

A class of AI systems that produces new content (text, images, audio, code, video) rather than classifying existing content. Generative AI is the umbrella. Large language models are the most familiar shape it takes, but image models, audio models, and video models live under the same umbrella.

Go deeper: Andrew Ng's free DeepLearning.AI course [Generative AI for Everyone](#) is the cleanest non-technical introduction. For an applied view, start with Module 1 of LevelUp Labs' [Generative AI Genius](#) free course.

Foundation Model

A model trained on broad data at scale, designed to be adapted to many downstream tasks rather than built for any one task. The term was coined by the Stanford Center for Research on Foundation Models in 2021. Today GPT, Claude, Gemini, Llama are all foundation models. You can use them directly or adapt them for a specific job through fine-tuning.

Go deeper: Stanford's paper [On the Opportunities and Risks of Foundation Models](#) is the definitional reference. Long, but the first twenty pages are the most cited definition in the field.

Large Language Model (LLM)

A foundation model specialized for language. It reads text, produces text. The “large” refers to the parameter count: today’s frontier LLMs have hundreds of billions of parameters, some approaching the trillion mark.

An LLM does not “know” facts the way a database does. It has learned to produce text that fits the patterns of its training data. The behavior that feels magical and the behavior that feels wrong both trace back to that one fact. Holding this in mind keeps a lot of later confusion at bay.

Go deeper: Anthropic’s research post [Mapping the Mind of a Large Language Model](#) is the most accessible serious read on what is actually happening inside these systems. Stephen Wolfram’s [What Is ChatGPT Doing and Why Does It Work](#) is the most useful longer explainer.

Tokens

The unit of text a model reads and writes. A token is usually a word, part of a word, or a character, depending on the tokenizer. “Evaluation” might be one token in one model and three in another. Two practical consequences: you pay by the token, and you have a limited number of tokens you can fit into a single request.

Go deeper: OpenAI’s interactive [Tokenizer tool](#) is worth five minutes. Paste in a page of your own content and watch how it splits.

Context Window

The total number of tokens a model can consider in a single request: your prompt, the conversation history, any retrieved documents, and the response it is generating. Frontier models in 2026 support context windows ranging

from around 200,000 tokens (Claude Sonnet 4.6 standard) up to 1,000,000 tokens (Claude Opus 4.7 with extended context, Gemini 2.5 Pro).

A larger context window does not mean the model will use every token well. Attention quality often degrades toward the middle and edges of very long contexts. This is called the “lost in the middle” problem and is a real production concern.

Go deeper: The original research paper, [Lost in the Middle: How Language Models Use Long Contexts](#). Anthropic’s [long context prompting guidance](#) is the best applied reference for getting more out of long windows.

Pretraining, Fine-Tuning, Post-Training

Three stages a model passes through on the way to being useful.

Pretraining is the expensive, general step: train on trillions of tokens of web, book, and code data. This is where the model learns the shape of language.

Fine-tuning adapts a pretrained model to a narrower task using a much smaller labeled dataset. Classic example: a general LLM fine-tuned on legal documents to be better at contract review.

Post-training is the umbrella for everything between raw pretraining and a deployable model: instruction tuning, reinforcement learning from human feedback (RLHF), safety tuning, reasoning training. Most of what distinguishes a “good” frontier model from a weaker one in practice is post-training, not pretraining scale.

Go deeper: The [InstructGPT paper](#) from OpenAI is the foundational read on instruction tuning and RLHF. For a more applied view of when fine-tuning is worth it, see LevelUp Labs’ [Fine-Tuning 101 guide](#).

1.2 From Text to Thought

A raw language model produces text that sounds confident. But confident text is not the same as a correct answer. The next layer of the field is about getting models to actually reason, not just speak fluently.

Chain-of-Thought

Asking the model to “think step by step” before producing a final answer. The counterintuitive result, first shown by a Google Brain paper in 2022, is that simply prompting a model to show its reasoning can dramatically improve accuracy on math, logic, and multi-step problems. Chain-of-thought is the foundational technique behind every reasoning technique that came after it.

Go deeper: Wei et al., [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#). A short paper, worth reading in full.

Reasoning Models

A newer class of models trained specifically to reason longer and more carefully before answering. OpenAI’s o-series and GPT-5’s reasoning mode, Anthropic’s Claude with extended thinking, Google’s Gemini 2.5 with thinking, and DeepSeek’s R-series are the current examples. They trade speed and cost for reliability on hard problems. The practical implication: for simple tasks, a standard model is faster and cheaper. For planning, math, code debugging, and agentic work, a reasoning model is usually worth it.

Go deeper: OpenAI’s [Learning to Reason with LLMs](#), Anthropic’s [Extended Thinking documentation](#), and the DeepSeek-R1 paper [DeepSeek-R1: Incentivizing Reasoning Capability in LLMs](#).

Multimodality

Models that accept more than text as input or output. GPT-5 with vision, Claude with vision, Gemini 2.5, and Qwen-VL can read images, charts, screenshots, and in some cases audio and video. The practical consequence for enterprise: you no longer need a separate OCR pipeline for most document work. A screenshot of a spreadsheet, a photo of a form, or a diagram can go directly into the prompt.

Go deeper: LevelUp Labs' [Introduction to Multimodal LLMs guide](#). For primary sources, see the [Gemini technical report](#) and Anthropic's [vision documentation](#).

1.3 Talking to the Model

Now the practical question: how to actually get the model to do what you want.

Prompting

The practice of crafting the input to a model so it produces the output you want. Prompting is one part craft (how you phrase things), one part design (what structure you give the model), and one part empirical testing (what actually works for your specific task).

Go deeper: Anthropic's [Prompt Engineering guide](#) and OpenAI's [Prompting guide](#) are the two best provider-authored references. For a structured course, Week 2 of LevelUp Labs' [Applied LLMs Mastery](#) walks through the full taxonomy.

System Prompt

A special kind of prompt that sits above the user's message and sets the model's role, rules, and scope. System prompts are sticky: they apply to every user turn in a conversation. This is where you write things like "You are a billing specialist. Do not claim you have applied a credit. Use only the tools provided."

Go deeper: Anthropic's [system prompts guide](#).

Few-Shot and In-Context Learning

Giving the model examples of the input-output pattern you want, directly inside the prompt. The model learns the pattern on the fly, without any fine-tuning. Few-shot learning is one of the most underrated techniques in the toolkit: three good examples often outperform a carefully tuned zero-shot prompt.

Go deeper: The original GPT-3 paper, [Language Models are Few-Shot Learners](#), introduced the term. For applied patterns, Anthropic's [multi-shot prompting guide](#).

Context Engineering

The current evolution of prompt engineering. Prompting focuses on the wording of a single instruction. Context engineering is the broader discipline of deciding what goes into the context window, in what order, with what structure. It includes system prompts, few-shot examples, retrieved documents, tool definitions, conversation history, and output format constraints, all orchestrated together. When people talk about "building an AI product" in 2026, they are almost always talking about context engineering more than model choice.

Go deeper: Andrej Karpathy's writing on context engineering (see his 2025 posts on X) captures the framing cleanly. For applied patterns, see Anthropic's post [Effective Context Engineering for AI Agents](#).

1.4 Beyond Single Calls

Everything so far assumes one request to a model and one response. Every interesting system in 2026 involves multiple calls, chained together in specific patterns.

Tool Use and Function Calling

Giving the model the ability to invoke external functions: search, database queries, calculators, custom APIs. The model decides when a tool is needed, formulates the call, reads the result, and incorporates it into its answer. A billing specialist in a customer-support system, for instance, might have tools like `get_billing_account`, `get_invoice_details`, and `read_billing_reference`. Tool use is the single most important capability that turns an LLM into something closer to an employee.

Go deeper: Anthropic's [tool use documentation](#) and OpenAI's [function calling guide](#).

Agent

A system where a language model uses tools in a loop to accomplish a goal, with the model deciding what to do at each step. Not every system that uses an LLM is an agent. An agent has to make runtime decisions about its next action based on what it observes.

Go deeper: Anthropic’s [Building Effective Agents](#) is the definitional essay. Read it twice. Also see LevelUp Labs’ [Agents 101 guide](#) for a structured walk-through.

Workflow vs Agent

Anthropic’s useful distinction. A **workflow** is a system where an LLM is embedded in a predefined path. You decide the steps; the model fills in the blanks. An **agent** is a system where the LLM decides the steps dynamically. Most production systems are workflows today, because workflows are easier to evaluate, control, and debug. Agents are more powerful but harder to trust. The honest answer for most enterprise problems: build the workflow first, add agentic flexibility only where it clearly earns its keep.

Go deeper: The distinction is made explicit in [Building Effective Agents](#).

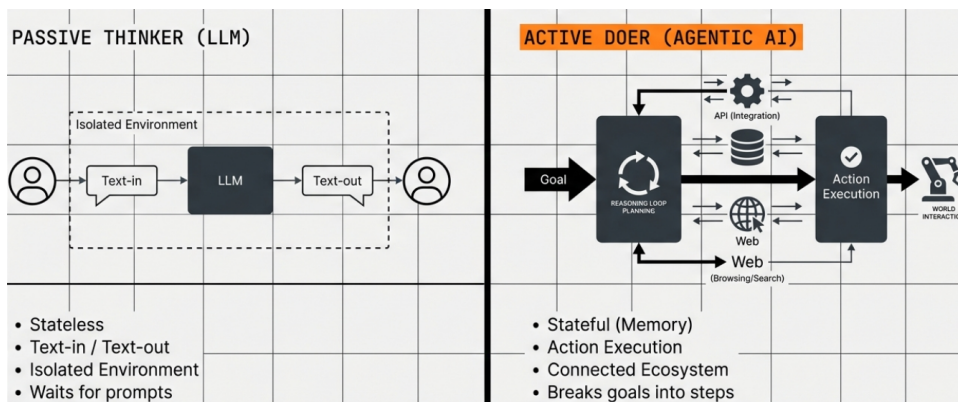


Figure 1: LLM vs Agentic AI

Visual from LevelUp Labs’ [awesome-generative-ai-guide](#).

Router and Classifier Patterns

A very common workflow shape: one LLM call decides which downstream specialist or path handles the request, and then the matching specialist takes

over. For example, a router might classify each support request as permissions, review-workflow, billing, or escalation, and then dispatch to the right specialist. Routers are cheap, fast, and one of the most effective ways to lift quality in production systems.

Go deeper: Anthropic’s [Building Effective Agents](#) calls this “routing” as one of five core workflow patterns.

Specialist and Sub-Agent

A downstream component that handles a narrower task than the orchestrator above it. In the customer-support example, the billing specialist has its own prompt, its own tools, and its own evaluation criteria, while the router only has to pick a category. Decomposing a system into specialists is one of the most reliable ways to make it both better and easier to evaluate.

Multi-Agent System

A system where multiple agents coordinate to solve a larger problem. Each agent has a role and often its own tools, and they communicate through messages or a shared state. Multi-agent systems are powerful but hard to get right: failure modes compound, debugging gets non-trivial, and the temptation to over-engineer is real. The current production consensus: start with one agent, add a second only when you have clear evidence you need one.

Go deeper: Anthropic’s [How we built our multi-agent research system](#) is the most honest writeup of the tradeoffs. The Google AI Co-Scientist paper, [Towards an AI co-scientist](#), is a good example of a serious production multi-agent system.

1.5 Knowing What You Don't Know

Models are trained on a snapshot of the world. They do not know your company's documents, today's news, or what your customer asked yesterday. Retrieval is how you fill that gap.

Retrieval

The general capability of looking something up and including it in the model's context before asking for an answer. Retrieval can pull from a document store, a database, a search engine, a knowledge graph, or a mix of all of them. We deliberately call it retrieval, not RAG, because the concept is broader than any single pattern.

Go deeper: LevelUp Labs' [3-day RAG roadmap](#) is the best fast path into the topic.

Semantic Search and Embeddings

Instead of matching keywords, convert text into a vector of numbers that captures its meaning. Then find other text whose vectors are close in that space. This is semantic search. The vectors are called embeddings. Embeddings are the connective tissue of most retrieval systems, and they are cheap to generate.

Go deeper: OpenAI's [embeddings guide](#) is the cleanest applied reference. For the research origins, see the [Sentence-BERT paper](#).

Vector Database

Infrastructure built to store and search embeddings at scale. Popular choices in 2026: Pinecone, Weaviate, Qdrant, pgvector (for teams already on Post-

gres), and LanceDB. Choice of vector database matters less than most vendors want you to believe. What matters is your retrieval strategy and your evals.

Go deeper: The [pgvector documentation](#) is worth reading even if you are not using Postgres, because it explains the core concepts cleanly.

Memory

Persisted state across turns or sessions. **Short-term memory** is the current conversation: previous turns held in the context window. **Long-term memory** is information the system chooses to remember across sessions: user preferences, past interactions, learned facts. The mechanics of long-term memory are still an active research area.

Go deeper: Anthropic's [memory tool documentation](#) and LangChain's [memory concepts](#).

1.6 Keeping It Honest

Once you have a model answering, the next problem is trust: whether the answer is good, whether it is truthful, whether it is safe. This section covers the vocabulary of that problem, which turns out to be most of the work in production.

Evaluation (Evals)

The practice of measuring how well your system performs on specific tasks, at scale, with a repeatable method. Evals are the one muscle worth building the most if you are shipping AI systems. Two broad families: **code-based**

evals (deterministic checks on format, schema, or exact matches) and **LLM-as-judge evals** (another model scores the output against a rubric). Most serious teams blend them.

Go deeper: Part 3 of this guide goes deep on each eval type. For the full mechanics with worked examples, [AI Evals for Everyone](#) is our companion course on this ground.

LLM-as-Judge

Using one language model to score another language model's output against a rubric. A common application: scoring whether a specialist's drafts match a brand voice. LLM judges are powerful because they scale and they can assess subjective quality. They are dangerous if they are not calibrated against human labels first. The rule: never trust a judge you have not calibrated.

Go deeper: The foundational paper is [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#). For applied guidance, our [AI Evals for Everyone](#) course has a dedicated module.

Hallucination

When a model produces confident text that is not grounded in reality. Hallucinations are not bugs you can fix by tweaking a prompt. They are a consequence of how LLMs work. You reduce them through retrieval, grounding, guardrails, and evals. You do not eliminate them.

Go deeper: OpenAI's research post [Why language models hallucinate](#) is a solid primer.

Grounding

Tying a model's output to a verified source, usually a retrieved document or a structured data record. A grounded answer can be traced back to its source. An ungrounded answer cannot. Grounding is the most effective defense against hallucinations in enterprise systems.

Go deeper: Google's [Vertex AI grounding documentation](#) and Anthropic's [citations documentation](#) both cover applied grounding patterns.

Guardrails

Safety checks that run before or after the model. **Input guardrails** filter user input: PII redaction, prompt injection detection, scope filtering. **Output guardrails** filter model output: hallucination checks, PII leakage detection, tone filtering, action boundary enforcement. Chapter 9 of this guide goes much deeper on the taxonomy and production patterns.

Go deeper: NVIDIA's [NeMo Guardrails documentation](#), Meta's [Llama Guard](#), and Microsoft's [Presidio](#) are the three most useful references.

1.7 Plumbing

Last, the layer nobody talks about at demos and everyone cares about in production.

Observability and Tracing

Watching what your system is actually doing in production. Every prompt, every tool call, every response, captured as a structured trace you can filter

and analyze. Observability is what turns a one-time evaluation into a continuous one. It is the foundation of CC/CD.

Go deeper: [Arize Phoenix](#) and [LangSmith](#) are the two most common platforms. For open standards, see [OpenTelemetry's GenAI semantic conventions](#).

Model Context Protocol (MCP)

An open protocol, originally published by Anthropic in late 2024, for connecting language models to external tools, data sources, and systems in a standardized way. Before MCP, every vendor had its own way of wiring up tools. MCP is fast becoming the USB-C of AI integrations. If you are building any non-trivial enterprise system in 2026, you are almost certainly going to encounter MCP.

Go deeper: The [MCP specification](#) and Anthropic's [Introducing the Model Context Protocol](#) announcement are the starting points.

Agents SDK

OpenAI's official framework for building agent applications, released in 2025. It handles the orchestration plumbing: the agent loop, tool calls, handoffs between agents, and built-in tracing. The package is called `openai-agents`. Other frameworks in the same space: LangGraph, CrewAI, LlamaIndex agents.

Go deeper: The [OpenAI Agents SDK documentation](#) and [LangGraph's documentation](#) are the two most mature references.

The Map, Assembled

That is the vocabulary. Read across it and you have the conceptual backbone of modern AI systems: a foundation model trained on text, talked to through prompts, extended through tools, grounded through retrieval, kept honest through evaluation, coordinated through protocols, observed through tracing.

Every chapter that follows drops deeper into one part of this map.

Chapter 2 steps back out to look at what enterprises are actually building with this language, and what is working in practice.

Chapter 2: What Enterprises Are Building

Now that you share a vocabulary, the next question worth asking is: what are other companies actually doing with these systems?

The honest answer has shifted a lot between 2023 and 2026. The early years of generative AI were dominated by demos and pilot projects. In the last eighteen months, the conversation has moved. More systems are in production, more teams are measuring ROI seriously, and a few clear patterns have emerged about what works and what does not.

This chapter summarizes what those patterns look like, drawing from public enterprise AI reports and from the work we do at LevelUp Labs. The goal is a map of the territory, so the choices you make for your own products land inside the zone where success is plausible.

2.1 The Shift From Pilots to Production

The numbers tell the clearest version of the story. McKinsey's annual State of AI survey shows the majority of enterprises now use generative AI regularly, a sharp climb from roughly a third of organizations just two years earlier. Menlo Ventures' 2025 State of AI in the Enterprise report put enterprise AI spending well into the tens of billions, with the steepest growth in production-grade systems rather than experiments. The direction is unmistakable: AI

spending is graduating out of the innovation budget.

What this has surfaced is a gap. Teams that succeeded in pilots did not always succeed in production. The work that turns a demo into a system your customers rely on is different work: evaluation harnesses, guardrails, observability, retraining loops. Chapters 6 through 18 of this guide are mostly about closing that gap.

Go deeper: [McKinsey's State of AI survey](#) (published annually) and [Menlo Ventures' perspectives](#) (they publish an annual State of AI in the Enterprise report).

2.2 Where the Value Is Actually Landing

The reports broadly agree on where enterprise AI is creating value today. Five categories dominate.

Internal knowledge and support. Internal Q&A systems, onboarding assistants, documentation bots, help desk routing. This category is the largest in most enterprise AI spend surveys because the ROI case is cleanest: fewer tickets, faster onboarding, less repetitive work for experts.

Document processing and extraction. Contract review, invoice processing, form parsing, compliance checking. This is a natural fit for document-heavy enterprises. Long documents are structured enough for LLMs to handle, and the errors are catchable.

Coding copilots. GitHub Copilot, Claude Code, Cursor, and a growing set of IDE-integrated agents. Developer productivity is the category with the clearest adoption numbers, with a meaningful share of engineering time now running through some form of AI assistance.

Customer-facing assistants. Support chat, sales qualification, onboarding flows. Less dominant than internal tools because the risk bar is higher, but

the category is growing as guardrails mature.

Research and analysis workflows. Literature review, competitive research, market analysis, investment diligence. Often built on top of retrieval systems.

Notice what is missing from this list. Fully autonomous agents doing complex multi-step work without human oversight are still rare in production. The honest story is that agentic patterns ship, but they ship with a lot of workflow scaffolding around them.

Go deeper: Anthropic’s [Economic Index](#) covers how Claude is being used across industries in aggregate. OpenAI publishes [customer stories](#) that are useful for shape-of-production reference.

2.3 The Patterns That Are Working

Across the production systems that stuck, a few design patterns show up again and again.

Augment first, automate later. The systems that shipped and stayed shipped tend to assist a human rather than replace one. The human stays in the loop, the system proposes and drafts, the human approves. Over time, the most trusted parts get automated one step at a time, as evals prove each step is safe to hand off.

Narrow scope, deep quality. Teams that picked a narrow problem (“draft responses to billing questions about plan changes”) and built a single great system for it shipped faster and with better quality than teams that tried to build “an AI assistant for everything.” Narrow scope makes evaluation tractable. Evaluation makes improvement possible.

Retrieval over fine-tuning for knowledge. For getting a model to answer with your organization’s specific information, retrieval almost always beats

fine-tuning in both cost and flexibility. Fine-tuning is for style, format, and tightly scoped task behavior. Retrieval is for knowledge.

Workflows before agents. The router-and-specialist pattern (covered in Chapter 11) is the most common production shape. It is structured, debuggable, and evaluable. Teams that tried to build free-roaming agents first often came back to workflows later.

Evaluation as durable infrastructure. The teams with production systems that actually improve over time built their eval harness as ongoing infrastructure, maintained continuously and visible to everyone. Teams that treat evals as a launch-time checkbox see quality drift within months. Chapter 6 is an entire chapter on this.

Go deeper: Anthropic’s [Building Effective Agents](#) is the clearest public articulation of the workflow-before-agent argument. The [a16z Enterprise AI report](#) has good sector-by-sector pattern breakdowns.

2.4 What Is Failing (and Why)

Almost every public enterprise AI post-mortem traces back to one of three failures.

No clear problem. The project started with “we should use AI” rather than “we have a specific pain, and AI might be the right tool.” Nine months later, there is a system with no adopter. This is the single most common cause of failure, and it is almost entirely preventable with thirty minutes of problem discovery upfront. Chapter 4 is about this.

No evaluation discipline. The team built something that looked good in demos, shipped it, and then could not tell why it was failing in production. With no evals, every bug becomes a guessing game. With no guessing game, the system drifts. With drift, trust erodes. The muscle Chapters 6 through 9 build is the muscle that prevents this.

Over-scoped agentic design. The team bet on a multi-agent autonomous system for a problem that needed a workflow. The failure modes compound, debugging gets expensive, and the project cannot ship. Chapters 10 and 15 cover when to use each shape.

The pattern across all three: each failure mode is a sign of skipping the boring work. The companies shipping production AI in 2026 are the ones who embraced the boring work early.

2.5 What This Means for Your Roadmap

A few practical takeaways for a product team at a document-heavy enterprise, reading this in the middle of 2026:

- Start with a narrow problem that has clear users and clear success criteria. If you cannot name who will use it and what changes when they do, go back to the problem.
- Build augmentation first. Let a human sit in the loop for the first version. Automate steps only after evals prove they are safe to automate.
- Pick retrieval over fine-tuning for anything knowledge-related.
- Budget more time for evaluation than for the demo. Demos are cheap. Evals are the work.
- Expect iteration. The first version will be wrong by design, and the second version will be better because you shipped the first.

The next chapter goes one layer deeper into one of these: how to actually pick which model to build on.

Continue to [Chapter 3: Models: How to Choose](#).

Chapter 3: Models: How to Choose

Picking a model is the question builders want to answer first and should often answer last.

The instinct to start by comparing GPT, Claude, Gemini, Llama, and the latest open-source frontier is understandable. It feels concrete. There are leaderboards. There are price-per-token tables. The problem is that the best model for your system depends on your task, your users, your latency budget, your cost ceiling, and the quality of your evals, none of which a leaderboard can tell you.

This chapter skips the “which model is better” question and gives you the decision framework that actually produces good model choices in production.

3.1 There Is No Best Model

The top-line benchmark (MMLU, HumanEval, GSM8K, whatever was trending last quarter) is a weak signal. Two reasons.

First, frontier models are close enough on general benchmarks that the gap is usually smaller than the variance from your prompt design. The gap between the top three closed models in any given month is often within a few points on most evals. The gap between a good prompt and a bad prompt on the same model can be twenty points.

Second, your task is not the benchmark. A model that dominates on MMLU may be average on your specific retrieval synthesis task. The only model comparison that matters is the one you run on your own evals.

Go deeper: Stanford's [HELM](#) project is the most serious public benchmarking effort and is worth bookmarking. [LMArena](#) (crowdsourced head-to-head comparisons) is a useful directional signal.

3.2 The Dimensions That Actually Matter

Frame model selection as a multi-dimensional tradeoff. These are the dimensions that move the decision in production.

Capability. How well does the model do on your task, measured on your evals. This is the dimension teams focus on almost exclusively and then realize later that other dimensions matter as much.

Latency. How long does a response take. Reasoning models can take 20 to 60 seconds. Standard models return in 1 to 3 seconds. For a customer-facing chat, latency is part of the product. For a background batch job, it is irrelevant.

Cost. Per-token input and output pricing varies by 10x across providers and by 100x between frontier and small models. A high-volume workflow can easily be the difference between \$200/month and \$20,000/month.

Context length. The maximum prompt size you can feed. In 2026, 200K tokens is standard, 1M is available from multiple providers. Larger does not mean better; attention quality often degrades past a certain point. But a larger window can simplify retrieval design.

Modality. Text only, vision in, audio in, tool use, structured output support. Match to your input types.

Reasoning. Whether the model is a reasoning model (Claude with extended thinking, OpenAI o-series, Gemini with thinking, DeepSeek-R1) or a standard model. Reasoning models are slower and more expensive but often dramatically better on multi-step problems.

Licensing and deployment. Closed (API only) vs open (Llama, Mistral, Qwen) vs hosted-open (Together, Fireworks, Groq). Matters if you have data residency requirements or need on-premise deployment.

Go deeper: Each provider publishes a model comparison matrix. Anthropic's [model selection guide](#), OpenAI's [model documentation](#), and Google's [Gemini model card](#) are the primary sources. For open models, [Hugging Face's leaderboard](#) is the standard reference.

3.3 A Decision Framework That Works

A simple rule that consistently produces good model choices: start capable, downsize where you can, escalate when you must.

Start capable. Prototype with the strongest model you can afford. Claude Opus 4.7 or Sonnet 4.6, GPT-5, Gemini 2.5 Pro. A capable model makes your prompts forgiving, your evals interpretable, and your early iterations fast. Debugging prompt problems while also debugging model limitations is twice the pain.

Downsize where you can. Once your system works end to end on a strong model, swap in smaller and cheaper models on the components where quality does not degrade. Simple classifiers, routing calls, and narrow specialists often perform nearly identically on Claude Haiku 4.5 or GPT-5 mini at a tenth the cost. The router-specialist pattern (Chapter 11) is perfect for this: use a cheap fast model to route, a capable model for the hard specialist.

Escalate when you must. If your evals plateau below the quality bar your product needs, and you have exhausted prompt engineering and retrieval

improvements, move up to a reasoning model. Reasoning models are overkill for most tasks but transformative for the few where they help. Multi-step planning, complex code, careful document analysis, decisions that require showing work.

Go deeper: Anthropic's [When to use Claude Opus 4 vs Claude Sonnet 4](#) post has useful guidance. OpenAI's [reasoning guide](#) explains when o-series models are worth the cost.

3.4 Reasoning Models: A Different Kind of Tool

It is worth spending a paragraph on reasoning models because their tradeoffs are sharper than most teams expect.

Reasoning models do not just think longer. They are trained specifically to reason through problems, make plans, and check their own work. The consequence: they are very good at problems that decompose into steps, and average to worse on problems that do not.

When to reach for a reasoning model: agentic planning, multi-step math or logic, code debugging across a codebase, careful document synthesis with cross-references, any task where you currently rely on chain-of-thought prompting and wish it were better.

When not to: single-turn classification, simple extraction, summarization, style transfer, chat. The extra cost and latency buy you nothing.

A classifier router is a standard-model task. A specialist that has to reason across many records (say, comparing line items across multiple invoices) is a good candidate for upgrading to a reasoning model.

Go deeper: OpenAI's research post [Learning to Reason with LLMs](#) and DeepSeek's paper [DeepSeek-R1: Incentivizing Reasoning Capability in LLMs](#)

via [Reinforcement Learning](#) are the two most useful public accounts of how reasoning models work.

3.5 Open vs Closed

The open-model story in 2026 is stronger than it has ever been. Meta's Llama 4 family, Alibaba's latest Qwen generation, and DeepSeek's V3 and R-series are genuinely competitive with frontier closed models on many benchmarks. They are fully controllable, cheap at scale, and deployable anywhere.

The reasons to choose open: data residency requirements, predictable per-inference cost at high scale, hard controls on what the model can and cannot do, fine-tuning for narrow tasks, avoiding lock-in.

The reasons to choose closed: the frontier for the most capable reasoning is still closed (Claude Opus 4.7, OpenAI's o-series and GPT-5, Gemini 2.5 Pro), API maturity is higher, tool use and structured output are more reliable, the operational overhead of self-hosting is zero.

Most enterprise systems in 2026 end up using both: a frontier closed model for the hardest parts, an open model deployed on an inference platform for the high-volume simple parts. The mix is cost engineering. Match each component to the cheapest model that passes your evals for it.

Go deeper: Meta's [Llama research](#) page. Hugging Face's [open leaderboard](#). Together AI, Fireworks, and Groq documentation for hosted-open options.

3.6 Your Evals Beat Any Benchmark

This chapter is full of signals you can use to narrow the choice. None of them replace the one thing that matters most: your own evals.

Write evals that reflect your product's real users and real failure modes. Run the candidate models against those evals. Decide based on results, not on marketing posts or benchmark tables.

Every chapter in Part 3 of this guide is about building these evals. If you have not read Part 3 yet, this is the chapter it most directly supports.

Continue to [Chapter 4: Problem-First Design, Revisited](#).

Chapter 4: Problem-First Design, Revisited

Every AI project that fails in production traces back to a design decision made months earlier. Almost always, that decision happened before anyone wrote a single prompt.

This is the chapter where we slow down and sit with the upstream work. Problem-First Design is the framework we use to ground every AI project before any prompt gets written. This chapter walks through it end to end. It surfaces the design traps we see most often, shows the questions that separate good AI product thinking from the rest, and gives you a simple scoping tool you can use on your next project.

4.1 Why Problem-First Matters More in AI

In traditional software, a weak problem statement still usually produces a working system. Engineers write code, the code does what it says, and the worst outcome is a product nobody wants. Expensive, but recoverable.

AI systems fail differently. A weak problem statement produces a system whose outputs look plausible but drift in ways you cannot predict. The failures are not deterministic. The evals do not catch what was never specified. The system ships, underperforms in a fuzzy way, and six months later nobody is sure whether the fix is a better prompt, a better model, a better dataset, or

a different problem.

Problem-First Design is the cheapest insurance you can buy against this failure mode. Thirty minutes of clean problem definition upfront saves months of unclear iteration later.

Go deeper: Anthropic's [Building Effective Agents](#) opens with the same thesis: "When building with LLMs, we recommend finding the simplest solution possible, and only increasing complexity when needed."

4.2 The Four Layers Framework

We use a four-layer framework at LevelUp Labs to structure any AI design conversation. It maps roughly to Marty Cagan's product framing, adapted for AI's specific failure modes.

Layer 1: The User and Their Pain. Who is the user, and what are they doing today that you want to change? Be specific. Named role, actual task, measurable pain. "Compliance analysts spend 90 minutes per filing checking disclosure completeness, and miss 3-5% of required items."

Layer 2: The Outcome. What changes when your system works? How do you know? "Analyst completes a filing check in under 20 minutes, with miss rate under 1%." Write the outcome before you write the system.

Layer 3: The AI Intervention. What does the AI system actually do? This is the narrowest useful definition of the system: the specific pattern (retrieve and check, draft and review, classify and route) and the specific capability (summarization, extraction, classification, generation). This layer is often bloated in failed projects. Narrow it until it hurts.

Layer 4: The System and Safety. What does the system need around it to be trustworthy? Evals, guardrails, observability, human oversight, rollback, audit trail. The production scaffolding.

The four layers should be written in order. Every layer should fit in a paragraph. If Layer 2 is vague, no amount of clever work on Layer 3 will save you.

Go deeper: [Marty Cagan's product discovery framework](#) is the parent idea. [Anthropic's engineering blog](#) has several applied writeups that follow a similar shape.

4.3 Six Traps We See Constantly

After thirty-plus enterprise implementations, six design traps come up over and over. Recognizing one is often enough to avoid it.

Tech-first thinking. “We want to use AI for X.” AI is the given, the problem is the search. Every AI project that leads with the technology ends up building something without a user.

Over-scoping. Three problems packed into one system because they feel related. The resulting system does none of them well and is impossible to evaluate. Split them.

Problem with no owner. A vaguely-agreed pain that nobody actually has to live with. Without a specific owner whose workflow is affected, you cannot define the outcome measurably. The project drifts.

Solutioning in the problem statement. “Build an LLM that uses RAG over our document store with a workflow agent on top.” A statement like this describes an architecture. The actual problem (who hurts, where, how much) is missing. Describe the pain first and let the architecture follow from it.

Skipping the human baseline. Building a system without ever measuring how long the task takes a human, how often they get it wrong, or where they struggle. Without a baseline, you cannot tell if the AI system is an improvement.

Confusing pilot success with production readiness. A pilot run with selected users on curated data tells you the system can work under ideal conditions. It does not tell you how it holds up when real users throw real edge cases at it. Most teams skip the hard transition.

Go deeper: McKinsey's [State of AI](#) reports cover these patterns with industry data, and Anthropic's [engineering blog](#) has multiple writeups on the "start with the simplest solution" principle.

4.4 A Scoping Template You Can Use

Before you kick off a design, fill in the following template. If you cannot, you are not ready to design.

Who is the user?

[Named role. Not "employees." A specific team with a specific task.]

What are they doing today, and how is it painful?

[Describe the current workflow. Time it. Measure the error rate.

Include a quote from one of them if you have one.]

What would change if the new system works?

[Describe the new workflow. How long does it take now.

What error rate is acceptable. What new things become possible.]

What is the single AI capability at the core?

[One sentence. One pattern. If you cannot say it in a sentence, the system is too complicated.]

What does "ready to ship" mean?

[Specific eval scores. Specific guardrail coverage.]

A human fallback. A rollback plan.]

What can go wrong, and what happens when it does?

[Enumerate failure modes. Map each to a recovery path.]

Fill this in collaboratively with the user and the engineering team in the room. The act of filling it in surfaces every unresolved disagreement before any code gets written.

4.5 Designing for Iteration

AI systems are not build-once artifacts. They are products that live in a loop: deploy, observe, learn, improve, redeploy. Problem-First Design has to accommodate this.

The practical implication: design your first version as the simplest thing that could work. Resist the urge to jump straight to the full end-state architecture. Ship the minimum in front of three users. Watch what breaks. Fix the top problem. Ship again.

The temptation to build the full system before shipping anything is strong, especially in enterprise where the cost of looking unpolished is real. Resist it. The cost of being wrong about the design is much higher than the cost of shipping something small.

The CC/CD framework (covered in Chapter 6) is the operational version of this principle. Continuous Calibration keeps your evals fresh as the world changes. Continuous Development keeps the system improving against them. Together, they turn a designed system into a working one.

Go deeper: Shreyas Doshi's writing on [product iteration loops](#) is useful background on shipping iteratively.

4.6 Problem-First Doesn't Mean Problem-Forever

A final caveat. Problem-First Design is a discipline for the first thirty minutes of a project, used well and then set aside. Once the problem is clear and the system is shipping, you should be spending most of your time on evaluation and iteration.

We raise this because we have seen teams cycle back into endless problem-definition meetings when the actual issue was slow execution. The value of Problem-First is that it ends clean, fast, and with enough clarity to unlock the build.

Good problem definition is like a map. It is worth reading carefully before the trip, worth keeping folded in your pocket while you travel, and worth not arguing about once you are moving.

Continue to [Chapter 5: Prompting and Context Engineering](#).

Chapter 5: Prompting and Context Engineering

There is a version of this chapter that lists twenty prompting techniques and calls it done. We are going to do something more useful.

The craft of getting a model to do what you want has shifted over the last two years. In 2023, it was prompting: the art of phrasing a single instruction well. In 2026, the field calls it context engineering: the discipline of deciding what goes into the model's context window, in what order, with what structure. Prompting is one piece of that. There are several others.

This chapter covers the parts of context engineering that matter most for product teams. Engineers, designers, and PMs all end up writing prompts and shaping context, so this chapter is for everyone.

5.1 The Six Parts of a Working Prompt

A prompt that consistently works has the same six parts, even when they are implicit. Making them explicit is the fastest way to improve prompt quality.

Role. Who is the model acting as. “You are a billing support specialist.” “You are a contract review assistant.” The role anchors tone, scope, and default behavior.

Context. The situation the model is operating in. The authenticated user, the current document, the relevant policy. Context is not static: in a real

system, it gets assembled at runtime from the user, the retrieval layer, and the tool results.

Task. What you want the model to do. One clear instruction. If you find yourself writing three tasks, consider splitting into three prompts.

Constraints. What the model must not do, or must do in a specific way. “Never claim you applied a credit.” “Cite the policy ID for every claim.” “Respond in under 200 words.”

Examples. Two or three cases showing input-output pairs for non-obvious patterns. Examples are the highest-leverage tool in the prompt toolkit for anything subjective or structured.

Output format. How the response should be structured. JSON schema, markdown template, a specific tag set. The output format is the interface between your model and everything downstream.

The order matters. Models weight the earliest and latest sections of a prompt more heavily. Most teams put role and context first, task and constraints in the middle, examples before the final output format reminder.

Go deeper: Anthropic’s [Prompt Engineering Overview](#) walks through each of these. OpenAI’s [Prompt Engineering guide](#) is a good companion read.

5.2 Prompting for Reasoning Models Is Different

Reasoning models (Claude with extended thinking, OpenAI o-series, Gemini with thinking) do not prompt the same way standard models do. Teams that carry over their old prompts often see worse results.

Three rules change:

Less hand-holding. Reasoning models do their own step-by-step thinking. Telling them to “think step by step” is redundant and sometimes harmful. Instead, describe the goal and the constraints, then let them reason.

More context, less structure. Where a standard model benefits from tight structure and explicit formatting, reasoning models often do better when given more room. Give them the full picture, not a tightly pre-processed summary.

Separate the what from the how. Tell the model what you want it to achieve and what success looks like. Skip the procedural instructions about how to get there.

The short version: for standard models you are scripting the task. For reasoning models you are briefing a capable specialist.

Go deeper: OpenAI's [Reasoning Guide](#) and Anthropic's [Extended Thinking documentation](#) are the primary references.

5.3 Context Engineering: The Bigger Picture

Once you move past single prompts, the discipline changes shape. You are no longer writing one prompt. You are assembling a context window dynamically, every time the model runs.

A real production request might include:

- A system prompt (the role and rules)
- A running conversation history (turns 1 through 7)
- Retrieved documents (three chunks, each 300 tokens)
- Tool definitions (the schema for the tools the model can call)
- Tool results (the output from a previous tool call)
- The current user message

Context engineering is about deciding what goes in, what stays out, in what order, with what structure, and how to compress or summarize when the window gets crowded. It is the difference between a prototype that works on short conversations and a system that holds up across a ten-turn customer support thread.

A few practical principles that show up in most production systems:

Recency matters. Models attend more strongly to recent content. Keep the most important context close to the end of the prompt.

Summarize when you compress. When the conversation gets long, do not just truncate. Summarize old turns into a running summary that preserves the key facts.

Retrieve what you need, when you need it. Do not dump everything into context at once. Pull the relevant chunks at the moment they are relevant.

Keep tool definitions stable. Frequently changing tool schemas confuse models. Stabilize the interface, iterate on the logic.

Go deeper: Anthropic's engineering post [Effective Context Engineering for AI Agents](#) is the clearest public writeup on this topic. LangChain's [context engineering guide](#) is a good applied companion.

5.4 Few-Shot Examples: Your Highest-Leverage Tool

If we could only teach one prompting technique, it would be few-shot examples. Including two to three example input-output pairs in your prompt consistently outperforms almost any other single intervention on subjective or structured tasks.

Examples work for three reasons. They teach the model the pattern without describing it. They calibrate tone and style, which are hard to describe in the abstract. They resolve ambiguity in the task definition implicitly.

A few rules for picking good examples: - Pick examples that cover the variety of real cases, not just the easy ones - Include one or two edge cases, not only the middle of the distribution - If your task has different categories, cover each category with at least one example - When the task changes, update the examples. Stale examples become a drag on quality

Go deeper: Anthropic's [multi-shot prompting guide](#) covers the mechanics. LevelUp Labs' [Applied LLMs Mastery Week 2](#) covers the full prompting taxonomy with examples.

5.5 Output Format: The Interface Nobody Talks About

The format of the model's output is the handshake between the AI system and everything else in your product. Get it wrong and downstream systems break, tests fail, and UX feels unreliable.

Three choices show up in most production systems.

Structured JSON. Best for anything a machine consumes next. All major providers support structured output with a schema, meaning the model is guaranteed to return valid JSON matching your shape. Use this whenever the next step is code.

Markdown. Best for anything a human reads directly. Clean, readable, supports lists and bold and code blocks. Useful as a default for chat interfaces.

Tagged sections. Best for output that mixes structured and narrative elements. Wrap each section in a tag like `<reasoning>...</reasoning>` and `<answer>...</answer>`. Easy to parse, flexible for the model.

Match the format to the next consumer, not to what feels natural to write.

Go deeper: Anthropic's [structured output documentation](#) and OpenAI's [structured outputs guide](#) are the two primary references.

5.6 When to Stop Iterating on Prompts

Prompt engineering has a ceiling. At some point, additional iteration stops producing gains and starts producing noise.

The signs you have hit the ceiling: - Small prompt edits produce inconsistent score changes on your evals - The model succeeds on most cases but consistently fails on a specific kind of edge case - You are adding more constraints and each one introduces a new failure mode elsewhere

When this happens, the fix is rarely a better prompt. More often it is one of: - Better retrieval (the model is missing context, not instructions) - A model upgrade (the task is at the edge of what this model can do) - A workflow restructure (the task needs to be split into two calls) - Fine-tuning (the task has a specific style or domain that prompts cannot reliably induce)

Recognizing the ceiling saves weeks of prompt-twiddling.

Go deeper: Anthropic's [prompt engineering overview](#) has a short section on when to move past prompting. The next chapter of this guide (Why Evals Are the Real Work) gives you the framework for knowing when you have hit the ceiling in the first place.

Continue to [Chapter 6: Why Evals Are the Real Work](#).

Chapter 6: Why Evals Are the Real Work

We are going to make a claim that matters. The most important skill for anyone shipping AI products in 2026 is evaluation. It beats prompting, agent design, and model selection as the single highest-impact muscle a builder can develop.

This is the opening chapter of Part 3, the deepest part of this guide. The next three chapters cover the core pieces of a working eval and safety stack: code-based evals, LLM-as-judge, and guardrails. Before we get to the mechanics, this chapter is about why the whole category exists and why teams who underinvest in it almost always ship AI products that underperform.

6.1 The 80/20 Flip

In traditional software, most of the work happens before shipping. You scope the feature, design the system, write the code, run tests, fix bugs, and deploy. Once the feature is in production, most software stays stable. Bugs get fixed, features get added, but the thing itself keeps doing what it was built to do.

AI systems flip this. The first version is relatively fast to build. A prompt, a model call, some retrieval, a UI. What takes time is what happens after. You watch real users interact with it. You spot patterns of failure. You study what the model got wrong and why. You adjust evals, fix prompts, update

the retrieval pipeline, and ship again. This loop runs continuously, often for the life of the product.

The work has not disappeared. It has moved. Where software spent 80% of the effort pre-deployment, AI spends 80% post-deployment. Most teams do not know this going in, and the ones who figure it out late spend months learning expensive lessons.

The implication for how you staff, schedule, and budget an AI project is significant. Treat evaluation and iteration as the core phase of the work, not as cleanup after the real building.

6.2 What Evals Actually Buy You

An eval harness is a set of test cases plus a way to score how your system does on them. It sounds simple. In practice, a good eval harness does four things that nothing else can.

It tells you if a change made the system better. Without evals, every prompt change is a vibe check. With evals, you have a number. You can reject changes that make things worse and keep ones that make things better.

It surfaces what is broken. A good eval suite covers the real edge cases. When the suite catches a failure, you know where to focus. Without one, you rely on users to tell you what is wrong, which is a slow and lossy feedback loop.

It creates a shared language across the team. Engineers, PMs, and designers can all look at eval scores and agree on what is working. No one has to trust the engineer's "it's better now." The scores do the talking.

It turns model upgrades from gambles into experiments. When a new model comes out, you do not guess whether to adopt it. You run it against

your eval suite. The numbers tell you.

The absence of evals produces a specific kind of stuck: the team ships, hopes, and iterates without signal. Every change feels uncertain. Every bug report produces a three-day investigation that could have been a ten-minute eval run.

6.3 The Two Questions Every Eval Answers

An eval, at its core, answers one of two questions about a specific case.

Did the system do the right thing? The output matched what we wanted. The classifier picked the right category. The summary captured the key points. The extraction pulled the correct values. This is the “correctness” question.

Was the way it did it safe and usable? The response was in the right tone. It did not reveal sensitive data. It did not claim an action it could not take. It was in the right format for the next consumer. This is the “quality” question.

Most production systems need both. Correctness alone misses the places where the system is technically right but the answer is unusable. Quality alone misses the places where the system sounds good but is actually wrong.

The taxonomy that follows in the next three chapters (code-based evals in Chapter 7, LLM-as-judge in Chapter 8, guardrails in Chapter 9) is a toolkit for answering these questions at scale, repeatably, across thousands of cases.

6.4 Evals Are Infrastructure, Not a Milestone

The single biggest mistake teams make with evals is treating them as a pre-launch checkbox. You write an eval suite before launch, run it once, watch

it pass, and ship. The evals then sit untouched for months while the system drifts.

Evals work best when they are infrastructure: always running, always visible, always evolving. Every new edge case you find in production becomes a test case in the suite. Every model upgrade gets scored. Every prompt change runs through the suite before going live.

The teams we work with who have invested in evals as infrastructure ship AI products that actually improve over time. The teams who treat evals as a milestone ship AI products that degrade quietly until someone notices.

The operational version of this is what we call the CC/CD framework. Continuous Calibration keeps the evals fresh. Continuous Development keeps the system improving against them. Together, they replace the one-time-launch model of software engineering with a living-system model.

6.5 A Companion Course

These four chapters are a condensed version of a topic we teach at much greater depth in [AI Evals for Everyone](#), a companion course we publish through our awesome-generative-ai-guide repo.

The course format (video lessons plus live sessions) lets us cover things a chapter cannot: worked examples on real datasets, the calibration workflow run end to end, and time to field questions. If anyone on your team wants the full treatment of eval design, it is there.

6.6 How to Read the Next Three Chapters

Chapter 7 covers code-based evals: deterministic checks that do not need human labels. Schema, format, exact match, length, URL validation, con-

tainment. Cheap, fast, unambiguous. The first eval category you should build.

Chapter 8 covers LLM-as-judge: using a model to score another model's output against a rubric. Powerful for subjective quality (tone, helpfulness, groundedness) that code-based evals cannot catch. Critical that you calibrate the judge against human labels before trusting it at scale.

Chapter 9 covers guardrails: the safety layer that runs before and after your model to catch bad inputs and bad outputs in production. Adjacent to evals, with a different job: evals measure, guardrails intervene.

Read them in order. They build on each other and together form a complete eval harness.

Continue to [Chapter 7: Code-Based Evals](#).

Chapter 7: Code-Based Evals

The first family of evals worth building is the cheapest and the most under-used. Code-based evals check whether a model's output meets a specific, deterministic criterion using plain code. No labels required. No human in the loop. No LLM judge. Just a function that returns true or false.

Teams skip these because they feel too simple to matter. That instinct is wrong. Code-based evals catch a surprising share of real production failures, they run in milliseconds, and they scale to millions of cases for essentially zero cost. Build these first, always.

7.1 What Code-Based Evals Can Check

The pattern is simple: the model produces an output, your code inspects it, and the code returns a pass or fail. Useful code-based evals fall into a few categories.

Schema validation. Does the output match the expected JSON schema? Are all required fields present? Are types correct? This is the highest-leverage code-based eval for any structured-output system. Modern providers support schema-constrained output so this rarely fails, but catching the rare failure matters.

Format validation. Does the output follow the expected format? Is the markdown valid? Does the response start with the required prefix? Does the category match one of the allowed values?

Exact match. Does the output exactly match an expected value? This is the canonical eval pattern for classification: a router picks a category, the eval checks if that category matches the labeled one.

Containment. Does the output contain a specific required string? The disclaimer text. The citation ID. The session marker. Fast to check, catches a specific class of regressions.

Length constraints. Is the output within the expected length? Too short (suggesting truncation) or too long (suggesting the model is running on) are both signal.

URL validity. For any system generating links, check that URLs resolve (a HEAD request returns a success code) and point to allowed domains.

Regex pattern match. For outputs with a known shape (dates, product IDs, invoice numbers), a regex check is a cheap correctness signal.

Count of things. Are there three bullet points when we asked for three? Two citations when we expected at least one?

The common thread: if there is a question you can answer by inspecting the output with code, there is a code-based eval waiting to be written.

Go deeper: The [Arize Phoenix documentation on code-based evals](#) walks through common patterns. LangSmith's [evaluation guide](#) covers similar ground with a slightly different framing.

7.2 When Code-Based Evals Are Enough

Code-based evals shine when the question has a crisp right answer. Classification. Extraction. Format compliance. Schema conformance. Routing. Tool call validation. For these, a code-based eval is not only enough, it is better than anything else. It is faster, cheaper, more consistent, and easier to debug than any LLM-based alternative.

A classification router is a perfect case. The task is: pick one of N categories. The expected category is labeled. The code-based eval checks if the predicted category matches. That single eval gives you a reliable baseline score and lets you measure whether prompt improvements actually help.

Before reaching for LLM-as-judge (Chapter 8), ask whether the question you are trying to answer is really subjective. Often the answer is no, and a five-line code function does the job.

7.3 Building Your First Code-Based Eval

The mechanics are simple. You need three things: a dataset of cases, a function that runs your system against a case, and a function that scores the output.

The dataset. A set of test cases. Each case has the inputs your system needs and the expected output (or the criterion the output has to meet). Start with 20 to 50 cases, cover the main patterns, and include a few edge cases.

The run function. Given a case, call your system (your LLM, your agent, your workflow) and return the output. This is the part of the eval that exercises the actual code path.

The score function. Given the output and the expected output, return true or false. This is your eval logic.

Run the suite, see the aggregate score, drill into failures. Fix. Rerun. Track the score over time.

You can use Arize, LangSmith, Promptfoo, or just a simple Python script to orchestrate the runs. The platform matters less than the discipline.

Go deeper: [Arize Phoenix](#), [LangSmith](#), and [Promptfoo](#) are the three most common choices. For a PM-friendly writeup, OpenAI's [evals guide](#) walks through the core concepts.

7.4 The Failure Modes Code-Based Evals Catch Best

Some failure types get caught by code evals more reliably than by any other method.

Schema drift. The model returned a field that was not in the schema, or skipped a required field. A schema validator catches it instantly.

Format regressions. A prompt change broke the output format in a subtle way. The containment or regex check catches it before any real user sees the bug.

Silent truncation. The model output was cut off mid-sentence because it hit the token limit. A minimum-length check or an end-of-response marker check catches it.

Category bleed. A classifier started predicting a new unexpected category because the prompt changed. An allowed-values check catches it.

Hallucinated links. A model generated a plausible-looking URL that does not resolve. A URL validator catches it.

Each of these is cheap to check and expensive to miss.

7.5 The Limits

Code-based evals cannot tell you whether the answer sounds right, whether the tone is appropriate, whether the summary captured the important points, or whether a reasoning chain is correct. These questions require judgment, either from a human or from an LLM judge.

Knowing this helps you split your eval suite correctly. Send everything that can be answered with code to a code-based eval. Reserve the LLM judge (Chapter 8) for the questions that actually require judgment.

A good rule: aim for 60 to 70% of your total eval coverage to be code-based. It catches the widest net of real failures at the lowest cost. The remaining 30 to 40% is where LLM judges and human review earn their keep.

7.6 What You Need in Your First Eval Suite

If you are starting from zero, here is the minimum eval suite we recommend:

- Schema validation on all structured outputs
- Allowed-value checks on all classifier-style outputs
- Exact-match or containment checks on any output that must contain specific required text
- Length bounds on any output with a specific target length
- URL validation on any output that contains links
- At least 30 test cases covering the main patterns and five edge cases

This setup runs in seconds, costs almost nothing, and catches a surprising percentage of real regressions. It is the fastest possible step from “we hope it works” to “we have a signal.” Build this before anything else.

Continue to [Chapter 8: LLM-as-Judge and Calibration](#).

Chapter 8: LLM-as-Judge and Calibration

Some of the most important questions about AI output cannot be answered by code. Is the tone warm or robotic? Is the summary faithful to the source? Did the response feel helpful to the user? These are judgment calls.

You can answer them with human reviewers. That is accurate but does not scale. You can answer them with code-based heuristics. That is fast but often wrong. Or you can use an LLM-as-judge: a language model that scores your system's output against a rubric.

LLM-as-judge is the second pillar of a serious eval suite. Used well, it gives you judgment at scale. Used poorly, it gives you confident noise. This chapter is about using it well.

8.1 When LLM-as-Judge Makes Sense

LLM judges are the right tool when three conditions line up.

The question requires judgment. Tone, faithfulness, helpfulness, completeness, empathy, adherence to a style guide. Things a human can evaluate in ten seconds but a regex cannot.

The scale is too big for humans. You have hundreds or thousands of cases to score. Humans could do it, but not on the cadence you need.

You can write a clear rubric. If you cannot articulate the standard in a paragraph, the LLM judge will not be able to apply it consistently.

If any of these three is missing, reach for something else. For small datasets, have a human review. For questions with crisp answers, use a code-based eval (Chapter 7). For questions where you cannot write a rubric, the problem is your rubric, not your tool.

8.2 A Brand Voice Judge: A Worked Example

A common shape: an LLM judge scores whether a customer-support specialist's drafts match a brand voice (calm, professional, concise, empathetic). The judge returns Good, Acceptable, or Poor plus one sentence of reasoning.

That example is a good reference for every LLM judge you will ever build. The components are always the same:

- A clear task definition (what is being judged)
- An explicit rubric with the labels and criteria for each
- A calibration set of labeled human examples
- A judge prompt that instructs the model
- A scoring mechanism that returns a label

The pattern is stable. The difference between a good judge and a bad one is all in the details.

8.3 Writing a Judge Prompt That Works

A judge prompt has to do three things at once: tell the model what to evaluate, give it the rubric, and constrain its output to something you can parse.

A judge prompt we have seen work well looks like this:

You are evaluating AI support responses for brand voice.

Return exactly two lines:

LABEL: <Good|Acceptable|Poor>

REASONING: <one sentence>

GOOD: Calm, professional, concise, and empathetic.

Does not claim unsupported write actions.

ACCEPTABLE: Mostly professional but generic, awkward,
slightly flat, or missing warmth.

POOR: Blaming, abrupt, defensive, off-brand, or claims
an action the system did not take.

A few principles embedded in that prompt.

Use a label taxonomy. Models do better with labels (Good, Acceptable, Poor) than with fine-grained scores (1 through 10). Numeric scores sound precise but drift between runs. Labels are blunter and much more stable.

Definitions for every label. What exactly is Acceptable versus Poor? If a human cannot tell, the model will not either.

Structured output. A format you can parse. The LABEL and REASONING pair is cheap to extract and easy to analyze.

Keep reasoning short. One sentence. Long chains of reasoning produce inconsistent labels across runs.

Go deeper: Anthropic's [evaluation guide](#) covers judge-style prompting. Arize's blog post [Evaluating LLMs with LLMs](#) is a good applied companion.

8.4 Calibration: The Step Nobody Skips Twice

An LLM judge you have not calibrated is a judge you cannot trust. Calibration is the process of checking how well the judge agrees with human labelers, before you deploy it at scale.

The workflow:

1. Pick 30 to 50 cases. Cover the range of quality (some obviously good, some obviously bad, some borderline).
2. Have two humans label each case with your rubric. Ideally, people who know the domain.
3. Where the humans disagree, discuss and resolve. If you cannot resolve a disagreement, your rubric is underspecified. Fix the rubric, relabel.
4. Run your judge prompt on the same 50 cases.
5. Measure agreement between judge and human consensus.

If agreement is 90%+, trust the judge and deploy. If it is 70 to 90%, iterate on the prompt, retry. If it is below 70%, the rubric or the task is not ready for a judge. Go back and sharpen the definition.

The 50-case calibration set also becomes your regression test for the judge itself. When you tune the judge prompt, run the whole set and make sure agreement stays above your bar.

This is the step teams skip when they are in a rush. The cost of skipping it is weeks of measuring the wrong thing.

Go deeper: The full calibration workflow is covered end-to-end in [AI Evals for Everyone](#) with worked examples on real datasets.

8.5 The Ways LLM Judges Fail

Even calibrated judges fail in specific ways. Knowing the failure modes keeps you from being surprised.

Position bias. When comparing two outputs side by side, judges systematically prefer the first one. Randomize the order.

Length bias. Judges often prefer longer responses because they look more complete, even when the shorter response is better. Control for this in your rubric.

Over-lenient scoring. Frontier models trained on helpfulness can err toward calling outputs Good when a human would say Acceptable. Your calibration step will surface this.

Sensitivity to phrasing. A small word change in the rubric can shift scores significantly. Once you have a calibrated prompt, treat it as a stable asset. Version it.

Model drift. When your judge model gets upgraded (say, Claude 4.6 to 4.7), scores can shift even when your rubric is unchanged. Recalibrate after every judge model change.

None of these disqualify LLM judges. They just argue for treating the judge as a measurement instrument that needs regular checks, not a set-and-forget tool.

8.6 Judges as Part of a Full Eval Suite

LLM judges are one pillar. A serious eval suite combines them with code-based evals (Chapter 7) and occasional human review.

A healthy mix looks something like this:

- **60 to 70% code-based evals.** Cheap, fast, deterministic. Run on every change.
- **20 to 30% LLM judge evals.** Handle subjective quality. Calibrated quarterly.
- **5 to 10% human review.** Spot-check a sample weekly. Surface unexpected failures. Feed new edge cases back into the other two buckets.

The humans do not scale. The code does not judge. The LLM judges fill the gap in between. Together, the three pillars cover the full space of what you care about.

8.7 When to Recalibrate

Recalibrate the judge whenever one of these happens:

- You change the judge prompt
- You change the judge model
- You change the rubric
- Your system starts producing outputs that look qualitatively different (new style, new use case, new failure mode)
- A quarter has passed since the last calibration

The cost is low (30 minutes of human labeling, a judge run, an agreement calculation). The benefit is high (confidence that your eval numbers are still measuring what you think they are).

Continue to [Chapter 9: Guardrails: Input and Output](#).

Chapter 9: Guardrails: Input and Output

Evals measure. Guardrails intervene.

Evals tell you how your system performs across a dataset, after the fact. Guardrails run in real time, on every request, and stop bad things from happening. They are the production safety layer.

This chapter covers what guardrails are, how they split across input and output, when to use which, and how to think about the tradeoff between strictness and user experience.

9.1 What Guardrails Are

A guardrail is a check that runs automatically as part of your system's request flow, and that can modify, block, or route the request based on its result. Three things distinguish guardrails from evals:

- They run on every request, in real time, in production
- They can take action on the request (block it, modify it, route it)
- They live inside the running system as part of the request path

Guardrails split naturally into two families: input guardrails (run before the model sees the request) and output guardrails (run after the model produces a response but before the user sees it).

9.2 Input Guardrails

Input guardrails are about stopping bad things from reaching the model. The main categories:

PII detection and redaction. Scan for social security numbers, credit cards, account numbers, and sensitive identifiers before they hit the model. Mask or redact before the prompt leaves your system.

Prompt injection detection. Catch attempts to override system instructions (“ignore your previous instructions and...”). Dedicated classifiers like Meta’s Prompt Guard or services like Lakera Guard detect these reliably.

Jailbreak detection. More sophisticated attempts to manipulate the model through role-play, encoded instructions, or multi-step social engineering. Harder to catch with simple patterns; requires a specialized classifier.

Sensitive data blocking. Prevent credentials, API keys, or proprietary documents from entering the model’s context when they should not be there.

Scope filtering. Reject requests that are out of scope for your system before they consume model time. A small classifier trained on in-scope versus out-of-scope examples handles this well.

Input sanitization. Strip HTML, scripts, and unusual unicode before the text reaches the model. Standard sanitization libraries do most of the work.

Each input guardrail trades a small amount of latency for a specific reduction in risk. In most enterprise systems, you want three to five of these active, tuned for your risk profile.

9.3 Output Guardrails

Output guardrails run after the model responds, before the user sees the output. Their job is to catch bad outputs before they become user-facing

problems.

Groundedness check. Does the response claim things supported by the source documents, or did the model invent them? A dedicated LLM call or a service like Galileo or Azure Groundedness Detection can check this.

PII leakage detection. Did the response include sensitive data from the context that should not reach this user?

Action boundary enforcement. Did the response claim an action the system did not actually take (“I have applied a credit”) when it is only supposed to draft a reply? A classifier or keyword check catches this.

Tone and brand voice filtering. Does the output match your organization’s voice? For regulated industries, this often includes disclaimer checks.

Domain boundary check. Did the response drift outside the scope of your system? A small classifier returns a safe fallback message when the response is out of scope.

Citation validation. If the response cites sources, verify that the sources exist and support the claim.

Tool call validation. If the model called a tool, verify the tool name is allowed and the arguments match the tool’s schema before executing.

Refusal mechanisms. When another guardrail triggers, you need a graceful decline that redirects the user helpfully rather than returning an error.

9.4 When Guardrails Replace Evals, and When They Supplement

Sometimes teams ask whether guardrails replace evals. They do not. The two tools do different jobs.

Evals measure whether the system is getting better or worse over time, on a representative dataset, with enough volume to be statistically meaningful. They run offline, they are the basis for experimenting with prompt and model changes, they feed into release decisions.

Guardrails run in real time on every request. They catch the specific bad outputs you do not want to reach users. They do not tell you whether the system is getting better.

A mature eval suite and a mature guardrail layer work together. The evals show you aggregate quality and catch regressions before ship. The guardrails catch the edge cases in production that slipped through.

If you had to pick one to build first, build evals first. Evals tell you how often guardrails will need to fire, which tells you which guardrails are worth building.

9.5 The Strictness Tradeoff

Every guardrail trades false positives against false negatives. Too loose and real problems slip through. Too strict and legitimate requests get blocked, frustrating users.

The right setting depends on the cost of each error.

For high-risk domains (financial advice, legal guidance, medical information), err on the side of strict. False positives that block a legitimate question are less costly than false negatives that produce a harmful response.

For lower-risk domains (internal knowledge assistance, internal drafting tools), err on the side of lenient. Over-blocking here creates a bad user experience with little upside.

Tune the threshold with real data. Run the guardrail on a month of real traffic, check the false positive and false negative rates, adjust until the ratio

matches your risk profile.

9.6 Production Patterns That Work

Three patterns are worth naming because they show up in most well-run production systems.

Layer guardrails in sequence. Input sanitization, then injection detection, then scope filtering, then the model. The cheap fast checks run first so that expensive ones only run on inputs that pass the easy gates.

Log every guardrail decision. Every time a guardrail fires (block, redact, modify), log it with the input, the decision, and the reason. This is your visibility into what the guardrails are catching and how often. It is also how you catch drift, when a guardrail that used to catch 1% of traffic starts catching 5%.

Build a fallback path. When a guardrail blocks a request, the user needs a graceful response. A templated message, a redirect to a human, an alternative path. Nothing frustrates users more than a system that refuses without explanation.

9.7 The Right-Sized Guardrail Stack

If you are starting a new system, here is a stack we would recommend you reach parity with before launch, drawn from what we see in production:

- Input sanitization (always on, cheap)
- PII redaction on input, for any system where users might share sensitive info
- Prompt injection detection, for any system exposed to external users
- Scope filtering, for any system with a focused purpose

- Groundedness check on output, for any retrieval-based system
- Action boundary enforcement, for any system with tools that can take actions
- Tone check on output, for customer-facing systems
- Domain boundary check on output, for any system that could drift
- Logging on every guardrail decision
- A graceful fallback path for when guardrails fire

You do not need all of these day one. You do need to know which ones apply to you, and the order in which you will add them as the system matures.

Go deeper: [NVIDIA NeMo Guardrails](#) is the leading open-source framework. [Lakera Guard](#), [Prompt Armor](#), and [Microsoft Presidio](#) are solid commercial options for specific guardrail categories.

Continue to [Chapter 10: From Single Calls to Agents](#).

Chapter 10: From Single Calls to Agents

The word “agent” has become so overloaded in 2026 that it almost means anything. In this chapter, we are going to define it narrowly, map the spectrum of what people actually build, and give you a straightforward rule for knowing when to move up the ladder.

This part of the guide is about agentic systems in general. This chapter is the map. Chapters 11 through 15 go deeper into the workflow and multi-agent shapes (11 and 15), and the core capabilities they rely on: tool use, retrieval, and memory (12, 13, 14).

10.1 The Spectrum

There is no binary between “system that uses an LLM” and “agent.” There is a spectrum. As you move up the spectrum, the model takes more decisions at runtime, the system gains flexibility, and the engineering cost goes up.

Single call. One prompt, one response. The simplest shape. Great for classification, extraction, summarization, rewriting. Everything in Chapters 5 through 9 applies.

Chained calls. Multiple prompts in a fixed order, output of one feeding the input of the next. The model does not decide the order. Still a workflow, just

a longer one. Good for tasks with multiple steps that you, the designer, know in advance.

Workflow. Multiple prompts, multiple paths, with branching logic that you design. A router decides which specialist handles the request. A planner decides which of several approaches to try. The model makes local decisions inside steps you defined.

Agent. The model decides the steps dynamically. Given a goal, it chooses what action to take next, observes the result, and decides the next action. The loop runs until the agent determines it is done. You specified the tools and the goal. The sequence is up to the model.

Multi-agent system. Multiple agents, often with different roles, coordinating to solve a problem together. The orchestration layer itself becomes complex.

The router-and-specialist pattern (Chapter 11) sits at “workflow.” The router is a classifier. The specialists are mostly single calls with tools. For most enterprise tasks, this is the right shape. Jumping to “agent” makes the system harder to evaluate without solving a real problem.

10.2 Why the Spectrum Matters

The higher up the ladder, the more powerful the system is in principle. Also the more fragile, the harder to evaluate, the more expensive to debug, and the more likely to fail in surprising ways.

Every step up costs something:

- Evals get harder. You can no longer test a fixed input-output pair. You have to test the whole trajectory.
- Latency goes up. More LLM calls means more wait time.
- Cost goes up. Each step is another model call.

- Debugging gets harder. A bug in an agent can be at any of five turns. Tracing through the decision chain takes real effort.
- Failure modes multiply. An agent can get stuck in loops, invoke tools incorrectly, or drift off-goal in ways a workflow cannot.

In exchange, you get flexibility. An agent can handle problems you did not anticipate. A workflow can only handle the cases you designed for.

The tradeoff is real. The question is when the flexibility is worth the cost.

10.3 A Simple Rule for Moving Up

Our rule, drawn from a lot of shipped systems and a lot of rebuilt ones:

Stay at the simplest level that handles 90% of your cases. Upgrade only when you have a clear reason.

Concretely:

- If a single call works, do a single call. Do not add a second call because it seems more sophisticated.
- If a chained workflow works, stay there. Do not add dynamic routing for an edge case that shows up once a month.
- If a workflow works, stay there. Do not turn it into an agent because agents sound cooler.
- Move up only when you have a specific class of cases the simpler shape cannot handle.

Anthropic's published guidance says the same thing. "When building with LLMs, we recommend finding the simplest solution possible, and only increasing complexity when needed." This is the consensus from teams who have shipped production agents and found that simplicity pays dividends.

Go deeper: Anthropic's [Building Effective Agents](#) is the single best public

writeup on this topic. Read it end to end. LevelUp Labs' [Agents 101 guide](#) is a solid shorter primer.

10.4 When Agents Are Actually the Right Shape

Having been cautious about when to reach for agents, here are the cases where agents genuinely earn their keep.

Long-tail task variety. If your system needs to handle hundreds of task variations that you cannot enumerate in advance, an agent's flexibility pays off. Research assistants, general-purpose copilots, coding agents.

Dynamic planning. If the right next step depends on information you only get after the first step (a search result, a tool response, an observation), an agent's loop structure fits naturally.

Open-ended user goals. If users can ask for anything and the system has to figure out how to help, agents handle this better than workflows.

Exploratory work. Data analysis, debugging, iterative problem-solving. Work where trial and error is part of the job.

For most enterprise use cases, the task is narrower than any of these. A billing draft specialist does not need an agent. A compliance checker does not need an agent. A customer support router does not need an agent. Workflows handle these faster, cheaper, and more reliably.

10.5 The Hybrid Pattern That Usually Wins

The pattern we see ship most often is a hybrid: a workflow that embeds small, scoped agents in the specific places they help, with the workflow providing structure everywhere else.

A customer-support system is a good example. The overall shape is a workflow (router plus specialists for billing, permissions, review-workflow, escalation). But the billing specialist, which has tool access, behaves agentially within its narrow scope: it decides which tool to call, what to do with the result, and whether to ask for confirmation. The agentic behavior is contained inside the specialist. The workflow wrapping it stays evaluable.

This hybrid shape buys you the flexibility of agentic behavior where it matters and the structure of workflows where it does not. Our rule of thumb: use workflows as the chassis, with small scoped agents embedded as components inside specific specialists.

10.6 Evaluating Systems as You Move Up

One last note on evals as the system gets more complex.

At the single-call level, you evaluate inputs and outputs. At the workflow level, you evaluate inputs, outputs, and which path was taken. At the agent level, you evaluate inputs, outputs, the full trajectory, the tools called, the order, the recovery from errors, and whether the goal was actually met.

The eval harness grows with the system. A team that ships an agent without an agent-aware eval suite ships a system they cannot improve. The cost of building the evals is part of the cost of choosing agents.

Chapter 16 goes deep on observability, which is how you actually capture agent trajectories in production so you can evaluate them.

Continue to [Chapter 11: Workflow and Router Patterns](#).

Chapter 11: Workflow and Router Patterns

Most production AI systems in 2026 are workflows. The designer decides the shape, the LLMs fill in the steps. This is the default architecture for enterprise AI, more common than single-agent and multi-agent systems combined.

This chapter covers the five most common workflow patterns, the router pattern in detail (because it is the most common), and the design decisions that separate workflows that ship and hold up from workflows that look good in demos and struggle in production.

11.1 The Five Workflow Patterns Worth Knowing

Anthropic's essay on Building Effective Agents names five patterns. These five cover most of what teams actually build.

Prompt chaining. Output of one LLM call feeds the input of the next. Useful when the task decomposes into steps that must happen in order. Example: extract key claims from a document, then check each claim against a source, then draft a summary of the verified claims.

Routing. One call classifies or decides, then the request is handed off to a specialized downstream handler. Example: classify a support ticket into billing, permissions, review-workflow, or escalation, then dispatch to the specialist

for that category. We use this customer-support routing example as a running case throughout this chapter and the next.

Parallelization. Multiple LLM calls run in parallel, either doing variations of the same task (for voting or diversity) or handling different aspects of the task at once. Example: run three different extractors on a document, then reconcile.

Orchestrator and workers. A central LLM plans the work, delegates to worker LLMs or tools, and synthesizes their outputs. Similar to an agent but with a cleaner boundary between planning and execution.

Evaluator and optimizer. One LLM produces output, a second LLM evaluates it, and the result is fed back for refinement. Useful for tasks with clear quality criteria where first-pass output is often not quite right. Example: draft a response, have a judge score it against the brand voice rubric, revise if needed.

Most real systems combine these. You route to a specialist, the specialist runs a chain, and a final evaluator checks the output before return.

Go deeper: Anthropic's [Building Effective Agents](#) is the clearest writeup of these five patterns with diagrams and code examples. Every product team should read it once.

11.2 The Router Pattern in Depth

The router pattern is the most common workflow shape in enterprise AI. It is also the one most likely to fail silently if built carelessly. Worth a closer look.

The pattern has three parts: a classifier that picks a category, a set of specialists (one per category), and a fallback for when classification fails.

The classifier. Usually a single LLM call. Takes the user input and returns a category label. In the customer-support example above, the router returns one of four categories (billing, permissions, review-workflow, escalation) based on the request.

The specialists. One handler per category. Each is optimized for its narrow task: its own prompt, its own tools, its own evaluation criteria. The specialist does not have to know about the other specialists.

The fallback. What happens when the classifier is uncertain, or when the classification is wrong. Options: escalate to human, default to a general handler, ask the user to clarify.

That is the shape. The details matter.

11.3 Designing a Router That Holds Up

A few design principles, each earned through watching routers fail in production.

Keep the category set small. Three to seven categories is the sweet spot. Below three, the router is not earning its keep. Above seven, classifier accuracy degrades and your specialists start overlapping.

Make categories mutually exclusive. If a request could reasonably fit two categories, you have a design problem. Either one category is too broad or you are missing a deciding criterion. Examples in the router prompt can handle borderline cases, but the categories themselves should be orthogonal.

Use a cheap fast model. The router does a simple classification task. It does not need a frontier reasoning model. Claude Haiku 4.5, GPT-5 mini, and Gemini 2.5 Flash are the right tools. Save the expensive models for the specialists.

Show the model examples. Few-shot examples are the single best router-quality intervention. Adding examples for the tricky cases (“The workflow is pending, but I need edit access” should be permissions) lifts accuracy dramatically.

Evaluate with code. Router outputs are categorical. Use an exact-match code-based eval (Chapter 7). No LLM judge needed. Run the eval on every prompt change.

Log every decision. Log what the router decided, what the confidence was (if you have it), and what the downstream specialist did. When quality drifts, the logs show you whether the classifier got worse or the specialists did.

11.4 When to Add Confidence and Human Review

Not every classification is equally certain. A well-designed router tracks confidence and routes the low-confidence cases differently.

Two ways to estimate confidence:

Ask the model. Have the router return the category and a confidence label (High, Medium, Low). The labels are subjective but surprisingly consistent if the prompt is stable.

Use logprobs. For providers that expose them, the probability the model assigned to the returned category is a usable confidence signal.

For low-confidence cases, the options are:

- Escalate to human review
- Default to a general-purpose handler that uses a stronger model
- Ask the user a clarifying question
- Do nothing (accept the miss) if the cost of misclassification is low

The choice depends on how expensive a miss is. For billing disputes, escalate. For FAQ routing, accept the miss.

11.5 Human-in-the-Loop Patterns

Workflows often include a human review step, and the design of that step matters more than most teams realize.

Three patterns show up most often.

Pre-action review. The AI drafts, the human approves before the action happens. Right for high-stakes actions: financial transactions, customer-facing emails, regulatory filings.

Post-action review. The AI acts, the human reviews a sample of completed actions. Right for high-volume low-stakes work: ticket routing, internal document categorization.

Escalation triggers. The AI handles most cases autonomously and escalates to human only when a specific condition fires. Right when you have a clear boundary between the cases the AI can handle and the cases it cannot.

Each pattern has different implications for throughput, cost, and user experience. Pick based on the cost of a wrong action and the volume of the workflow.

Go deeper: LangGraph's [human-in-the-loop documentation](#) walks through the mechanics in code. Anthropic's [engineering blog](#) has posts on production agent patterns worth browsing.

11.6 Workflow Composability

A well-designed workflow is composable. Each step (router, specialist, tool, evaluator) should be independently testable, independently swappable, and independently improvable.

The test: can you swap out a specialist without changing anything else? Can

you add a new category without breaking the existing ones? Can you upgrade the router model without the specialists caring?

If yes, you have a workflow that will hold up as the product evolves. If no, you have a system where every change risks breaking something unexpected. Workflows that age well are built from clean interfaces between components.

The practical implication: design the contracts (the inputs and outputs of each step) before you write the implementation. Your components will stay clean. Your future self will thank you.

11.7 The Workflow Shape That Works for Document-Heavy Use Cases

For document-heavy enterprise work (compliance checks, filing reviews, policy adherence), a workflow shape that consistently works looks like this:

1. Classify the document type and the review intent
2. Retrieve the relevant rules or policies
3. Run an extraction pass to pull the key facts
4. Run a verification pass comparing facts against rules
5. Flag issues with structured output
6. Generate a human-readable summary
7. Route to human review if any high-severity flag appears

Each step has its own prompt, its own eval, and its own guardrails. The workflow is rigid enough to be reliable and flexible enough to accommodate new rule types as they come up. This is the shape most production document-review systems converge on eventually.

Continue to [Chapter 12: Tool Use and Actions](#).

Chapter 12: Tool Use and Actions

A language model that cannot use tools is limited to what is in its training data. A language model with tools is an employee.

This is the chapter where the system stops being a glorified autocomplete and starts being something that can actually do work on your behalf. Tool use is also where the risk profile of the system changes: a model that can call APIs can also call them wrong, and a model that can take actions can also take the wrong action.

This chapter covers tool design, the progression from read-only to write-action, permission patterns, and error handling. Engineers will recognize much of this; PMs and designers will find the decision framework that governs what gets built, in what order.

12.1 How Tool Use Actually Works

The core pattern is simple. You define tools (functions the model can call) with names, descriptions, and input schemas. You expose these definitions to the model in the prompt. When the model decides a tool is needed, it returns a structured tool call instead of plain text. Your code executes the tool, returns the result, and gives it back to the model. The model uses the result to continue the response.

The loop can run multiple times in a single user-facing response. The model

calls one tool, sees the result, calls another, sees that result, and finally produces its text for the user.

Consider a billing specialist in a customer-support system, with three tools: `get_billing_account`, `get_invoice_details`, and `read_billing_reference`. The model decides when to call each, the framework executes them, and the specialist uses the results to draft a grounded response.

Go deeper: Anthropic's [Tool Use documentation](#) and OpenAI's [Function Calling guide](#) are the two primary references. Both walk through the mechanics with code examples.

12.2 Designing a Good Tool

Not every tool is created equal. A well-designed tool is intuitive to the model and hard to misuse. A poorly designed tool produces wrong calls, wasted tokens, and failure modes you did not anticipate.

Five principles for tool design.

Name it for what it does, not what it is. `get_invoice_details` is better than `invoice_api_v2`. The model picks tools based on names and descriptions; make both self-explanatory.

Write a description that explains when to use it. Not just what the tool does, but when the model should reach for it. “Use this when the user asks about a specific invoice.” Help the model route.

Keep the input schema tight. Only the parameters that are actually needed. Optional parameters are fine. Unclear parameters are not.

Design the output to be useful. The model is going to read the output and use it. Structured, concise, with clear field names. Avoid internal codes and database jargon.

Make failures legible. When the tool fails, return a clear error message the model can understand and act on. “Invoice ID not found” is better than “ERR-4011”.

A good tool reads almost like a function another developer would write for a colleague. Clear name, clear purpose, clear inputs, clear outputs, clear errors.

12.3 Read-Only Tools First

When adding tools to a system, always start read-only. Read-only tools fetch information: look up an account, search a document, retrieve a record, check a status. They do not change anything in your systems.

Read-only tools are safe to let the model use freely. The worst case is a wasted call or a wrong lookup, neither of which has persistent consequences.

Write-action tools are different. They change state: create a ticket, send an email, update a record, apply a credit. When the model gets one of these wrong, something in your system is now wrong.

The progression we consistently recommend:

1. **Read-only tools, autonomous.** The model can call these freely. Most production systems live here for months before moving up.
2. **Write-action tools with draft-then-confirm.** The model proposes the action, a human approves before it runs.
3. **Write-action tools with guardrails.** The model runs the action autonomously but only within tight boundaries (specific record types, specific value ranges, specific user contexts).
4. **Write-action tools, autonomous.** The model runs actions fully on its own. Reserved for actions with low blast radius and high volume.

Most enterprise systems ship safely at level 2. Level 3 requires serious investment in evals and guardrails. Level 4 is rare for good reasons.

12.4 Permissioning: The Part Teams Skip

A model with a tool has the same permissions as the code running the tool. If the tool can read any account, the model can read any account. If the tool can refund any transaction, the model can refund any transaction.

This is almost always wrong. You want the model to have permission scoped to the current user's account, transactions, and documents only. Any wider scope is a vulnerability.

The pattern: scope the tool's permissions to the authenticated user's context, passed in at runtime. The tool's implementation enforces the scope. The model cannot bypass it, even if its prompt tells it to.

In the billing-specialist example, the tools are scoped to a specific `authenticated_account_id`. The tool implementation checks it. Even if a user asks the model to look up a different account, the tool refuses. The model's prompt cannot override the tool's code.

This is the correct pattern. Always.

12.5 Error Handling When Tools Fail

Tools fail. APIs time out. Records do not exist. Permissions deny. Networks glitch. A system that uses tools has to handle all of these gracefully.

Two principles matter here.

Surface the error to the model. Return a structured error message to the model, the same way you would return a successful result. Let the model decide how to handle the error (retry, ask the user, escalate).

Limit retries. Without a cap, a model facing a failing tool can loop indefinitely. Cap retries per tool call and per full request. When the cap is hit, return a clean fallback response.

The common anti-pattern is silently swallowing tool errors and returning “something went wrong” to the user. That user-facing message does not tell the user what happened, and the model never gets a chance to recover gracefully.

12.6 A Reference Tool-Use Shape

The billing-specialist example above is a good reference implementation for the most common production shape.

- Read-only tools only (level 1 in the progression above)
- Tools scoped to the authenticated user via the prompt and implementation
- Clear tool names and descriptions
- Structured, concise outputs
- Legible error messages
- The model drafts a response, does not claim actions it did not take
- A guardrail enforces that no “I applied a credit” type claims appear in output

If you start with this shape for any tool-using specialist in your product, you are starting in a good place.

Go deeper: The Arize blog post [The Essential Guide to Tool Use](#) covers the end-to-end pattern. LevelUp Labs’ [Agents 101 guide](#) has a section on tool-using agents that complements this chapter well.

12.7 When to Add a New Tool

A common mistake: adding too many tools too fast. Each tool is a thing the model might call, which means each tool is a way the system can fail. More tools means more to evaluate, more to maintain, more failure modes.

The rule we use: add a tool only when there is a clear task the model cannot complete without it, and at least 20% of your user requests require that task.

If you add a tool the model rarely calls, you have spent engineering effort on something that almost does not exist in production. If you add a tool the model calls confusingly (sometimes uses it when it should not, sometimes ignores it when it should), you have introduced a failure mode.

Tools are a commitment. Treat each one like a new feature.

Continue to [Chapter 13: Retrieval](#).

Chapter 13: Retrieval

Language models are trained on a snapshot of the world. They do not know your company’s documents, today’s news, or what a specific customer asked last week. Retrieval is how you give the model access to things it was not trained on.

The broader term for this capability is RAG, for retrieval-augmented generation. We prefer “retrieval” in this chapter because the concept is bigger than any single pattern, and the tooling has evolved well past the original 2020 formulation. This chapter covers retrieval as a product capability: what it is, what choices matter, and what separates a retrieval system that works from one that produces confidently wrong answers.

13.1 What Retrieval Actually Does

The shape is simple. When a user asks a question, your system looks up the most relevant information from a knowledge source, includes that information in the model’s context, and then asks the model to answer based on what was retrieved.

Three things happen at query time:

1. The query is turned into a search (keyword, semantic, or both)
2. The top matches are fetched from your knowledge store
3. The matches are included in the prompt to the model, which generates the answer

The quality of the final answer depends on the quality of the retrieval. If the right document is not in the top matches, the model cannot answer correctly, no matter how smart it is. Retrieval is the part of the system most teams underinvest in.

Go deeper: LevelUp Labs' [3-day RAG roadmap](#) is the fastest way to get the full shape. For a provider's applied guide, Anthropic's [Contextual Retrieval post](#) is worth the read.

13.2 Semantic Search, Keyword Search, and Why You Usually Want Both

Two ways to find matches exist.

Keyword search. Match the words in the query to the words in the documents. Fast, precise when the query uses the same vocabulary as the document, useless when it does not. The user searches “revenue recognition policy” and gets a document that uses the exact phrase. If the document calls it “accounting of income,” keyword search misses.

Semantic search. Convert the query and the documents into embeddings (vectors of numbers that capture meaning). Find documents whose vectors are close to the query's vector. Works even when the words are different. The user searches “how do we account for unearned income” and finds the revenue recognition policy.

Each has blind spots. Semantic search can surface documents that are thematically close but not actually relevant. Keyword search misses paraphrases.

The production answer is usually hybrid retrieval: run both, combine the results, rerank them together. Most retrieval quality improvements come from tuning this combination.

Go deeper: Pinecone's [Hybrid Search explainer](#) and Weaviate's [Hybrid Search documentation](#) cover this well. OpenAI's [embeddings guide](#) is the best applied starting point for semantic search.

13.3 Chunking: The Decision Most Teams Get Wrong

Before you can retrieve documents, you have to split them into pieces. A 50-page document is too big to embed as a single vector. You break it into chunks, embed each chunk, and retrieve at the chunk level.

How you chunk matters a lot. Chunks that are too small lose context. Chunks that are too big drown the signal. Chunks that split mid-sentence or mid-section produce garbled retrievals.

A few rules that produce reliable chunking.

Respect structure. Chunk on natural boundaries: sections, paragraphs, headings. Never split in the middle of a sentence or a table.

Size the chunks to the task. For Q&A on documents, 200 to 500 tokens is usually right. For document summarization, larger chunks. For code search, smaller.

Include context with each chunk. Every chunk should carry enough context to make sense on its own. A chunk from the middle of a contract should include the section title and the parent section's heading. Anthropic's Contextual Retrieval approach adds a short summary to each chunk at index time, which lifts retrieval quality significantly.

Overlap strategically. Overlapping chunks by 10 to 20% catches information that straddles a boundary. Too much overlap inflates your index; too little loses information.

Chunking is not glamorous work. It is also the single highest-leverage place to improve retrieval quality for most teams.

13.4 Ranking and Reranking

When you retrieve, you get a ranked list of matches. The top five matches are not always the best five to send to the model. A second pass (reranking) often improves quality.

The pattern: retrieve 20 to 50 candidates using your primary search, then run a reranker (a cross-encoder or a specialized reranking model) on the candidates to pick the top 3 to 5 for the prompt.

Rerankers are smaller, slower models that look at the query and each candidate together and score the match more carefully than a vector similarity can. Cohere, Voyage, and Jina all offer good reranking APIs. Open-source options exist too.

Reranking is one of the cheapest upgrades in the retrieval stack. If you are not doing it, add it.

Go deeper: Cohere's [Rerank documentation](#) explains the concept well. Pinecone's [blog on reranking](#) has good applied examples.

13.5 Evaluating Retrieval

Retrieval has its own eval suite, separate from the overall system eval.

The three retrieval-specific metrics:

Precision at K. Of the top K retrieved documents, what fraction are actually relevant to the query? A precision-at-5 of 0.8 means 4 out of 5 retrieved docs were relevant.

Recall at K. Of all the documents that are actually relevant, what fraction did we catch in the top K? A recall-at-5 of 0.6 means we found 60% of the true relevant docs.

Faithfulness. When the model generates an answer, does the answer actually use the retrieved context? Or did the model ignore the retrieval and hallucinate? This is usually checked with an LLM judge.

Build these evals alongside your main evals. When quality drops, you want to know whether the retrieval got worse or the generation did. Without separate retrieval evals, you cannot tell.

13.6 Retrieval Patterns Beyond the Basics

A few retrieval patterns that have become mainstream in 2025 and 2026.

Contextual retrieval. Adding a short summary or contextual label to each chunk before embedding. Anthropic's published version of this lifts retrieval accuracy by a measurable margin without changing the underlying infrastructure.

Agentic retrieval. The model decides what to search for, iterates on the search if the first pass is insufficient, and terminates when it has enough. More flexible than static retrieval, more expensive per query. Usually worth it for complex research tasks.

GraphRAG. When your knowledge has rich relationships (entities, references, citations), build a knowledge graph and retrieve along its edges. Microsoft's GraphRAG implementation is the most public reference.

Multi-vector retrieval. Instead of one embedding per chunk, store multiple (title embedding, summary embedding, full-text embedding). Search across all of them. Helps when queries vary in specificity.

Caching. Remembered retrievals for common queries. Drops latency and cost. Anthropic's prompt caching feature does this at the model-call level; application-level caching does it at the retrieval level.

None of these are required. All of them are worth knowing about for the moment you need one.

Go deeper: [Microsoft GraphRAG documentation](#) for the graph approach. Anthropic's [prompt caching documentation](#) for caching patterns. LlamaIndex's [advanced retrieval documentation](#) is a good overview of the state of the art.

13.7 What to Do Before Shipping a Retrieval System

A retrieval system is not ready until all five of these are true:

- Chunking respects natural document structure
- Hybrid search is in place (semantic + keyword)
- Reranking runs on the top candidates
- Separate retrieval evals measure precision, recall, and faithfulness
- A fallback exists for when retrieval returns nothing relevant (don't guess; tell the user what you could not find)

If any one is missing, the failure mode is predictable. Know which one you are missing before you launch.

Continue to [Chapter 14: Memory and Long-Running Agents](#).

Chapter 14: Memory and Long-Running Agents

A model with no memory lives in a single moment. Every turn starts fresh, every session forgets the last. For many use cases, that is fine. For many others, it is a dealbreaker.

Memory is the set of patterns that let a system remember things across turns, across sessions, across users. It is also one of the most actively evolving areas of AI engineering in 2026. This chapter covers the basics clearly, flags the places where the field is still working things out, and gives you a framework for knowing when memory is worth adding and when it is not.

14.1 Short-Term and Long-Term Memory

The two main kinds of memory, same as in human cognition.

Short-term memory is the current conversation. The last few turns, held in the context window. The model sees them on every turn and uses them to respond coherently. This is a basic expectation for any chat system.

Long-term memory is information that persists across sessions. The user's preferences, the facts learned in previous conversations, the decisions already made. Long-term memory is what lets a system feel continuous over weeks and months.

Short-term memory is solved. You include prior turns in the prompt, sometimes summarized if the conversation gets long. Long-term memory is where the interesting design work happens.

14.2 When Memory Is Worth Adding

Memory is not free. Every memory system adds engineering complexity, new failure modes (stale memory, incorrect memory, memory leaks between users), and evaluation overhead. Before adding it, be honest about whether you need it.

The cases where long-term memory is worth the cost:

Personalization over time. The user's preferences change the system's behavior meaningfully. A writing assistant that remembers the user's voice. A research agent that remembers which sources the user trusts.

Multi-session projects. The user works on the same thing across many sessions and the system needs to pick up where they left off. A coding agent that remembers the project structure, a legal drafting assistant that remembers the client matter.

Cumulative learning. The system genuinely improves for a user the more it sees them. Rare, but real for some specialized copilots.

The cases where memory is tempting but not worth it:

Single-session tasks. A support bot answers one question per session. Memory buys almost nothing and adds risk.

Tasks where fresh context is safer. Compliance review, audit, anything where drift from prior conversations could introduce subtle errors.

Tasks where retrieval does the job. If you can look up the relevant history at query time (retrieval), you do not need a persistent memory layer. Retrieval is often simpler and safer.

When in doubt, start without memory. Add it when you have a specific use case that proves it is worth the work.

14.3 Short-Term Memory Patterns

The patterns for short-term memory are stable and well-known. Three that cover most cases.

Full history. Include every prior turn in the prompt. Works for short conversations. Breaks when the context gets too long or too expensive.

Sliding window. Include only the last N turns. Drops the earliest turns when the window fills. Simple, cheap, loses early context that might have mattered.

Summarized history. When the conversation gets long, summarize the oldest turns into a running summary and keep only the recent turns verbatim. Preserves the gist of earlier context without blowing the budget. This is what most production chat systems do.

The right choice depends on conversation length, cost sensitivity, and how much early context matters for later answers.

14.4 Long-Term Memory: The Building Blocks

Long-term memory systems have three components: storage, retrieval, and write policy.

Storage. Where the memory lives. Usually a database or vector store, sometimes both. Each memory entry includes the content, a timestamp, and metadata about who and what it relates to.

Retrieval. How you surface relevant memories at query time. Almost always semantic search on the query, against the user's memory store. The top

matches get included in the prompt.

Write policy. When and what to remember. This is the hardest part. Remember everything and the memory store bloats with noise. Remember selectively and you miss things that mattered.

A few common write policies:

- Remember explicit user requests (“Remember that I prefer short summaries”)
- Remember decisions made in the conversation (“User chose Template A for the contract”)
- Remember identifiers and references (“User’s project is named ‘Atlas’”)
- Remember summaries of each session at session end

Each of these is a design decision. Each has tradeoffs.

Go deeper: Anthropic’s [Memory Tool documentation](#) shows their recommended pattern. [LangGraph’s memory documentation](#) walks through the building blocks. [Letta \(formerly MemGPT\)](#) is the most mature open-source memory system and a good reference implementation.

14.5 Memory Evaluations

Memory systems need their own evals, distinct from your main evals.

Three questions your memory evals should answer:

Is the right memory being retrieved? When the user asks about their project, does the system pull up the right project details? Precision and recall, same as retrieval evals (Chapter 13).

Is the memory still accurate? Memories can become stale. The user changed their preference, moved their project, updated a fact. A memory system that returns outdated memories is worse than no memory at all.

Is the memory write policy capturing what matters? A week later, can you answer questions about the conversation using only the memory entries? If no, the write policy is missing things.

The evaluation cadence: weekly or monthly, on a rolling set of recent conversations. Most teams underinvest here and pay for it later when users complain that the system keeps “forgetting” things they explicitly said.

14.6 Long-Running Agents

Adjacent topic worth covering: long-running agents. Systems that work autonomously on a task for minutes, hours, or longer.

A long-running agent is typically an agent plus checkpointing plus a mechanism for human intervention. Key design elements:

Checkpointing. The agent saves its state at natural pause points (after each major step) so it can be paused, inspected, and resumed. LangGraph’s checkpoint primitives are the most mature public implementation.

Time and cost budgets. Long-running agents can burn tokens fast. Set a budget (time, cost, number of steps) and have the agent terminate gracefully when it hits the budget.

Observability. You need to see what the agent is doing at each step. Chapter 16 covers this.

Human intervention. At any point, a human should be able to pause, inspect, and redirect the agent. Long-running agents without this always eventually take an unwanted action.

Long-running agents are one of the hardest categories of AI system to get right. They are also where the most ambitious 2026 systems live. Build them with extra care.

Go deeper: [LangGraph's long-running agents documentation](#) covers check-pointing well. [OpenAI's Agents SDK documentation](#) includes long-running patterns.

14.7 Memory Gotchas Worth Knowing

Three failure modes that show up repeatedly in memory systems.

Cross-user leakage. A memory from User A's session surfaces in User B's session. The absolute worst failure mode for memory systems. Always scope memory to a user at write time, retrieve scoped to the same user at read time, and test cross-user isolation explicitly.

Memory poisoning. A user plants a memory that manipulates the system's future behavior. Less studied than prompt injection but a real concern for open systems. Treat memory writes the same way you treat user input: sanitize, scope, verify.

Runaway memory growth. A system with no memory eviction accumulates everything, and the vector store becomes a graveyard of stale entries that hurt retrieval quality. Design eviction from the start: time decay, user-initiated deletion, relevance scoring.

Knowing these going in saves months of surprise debugging.

Continue to [Chapter 15: Multi-Agent Systems](#).

Chapter 15: Multi-Agent Systems

The temptation to build a multi-agent system is strong in 2026. Multi-agent reads as sophisticated. It reads as capable. Every AI conference has a talk on it. Every framework has a multi-agent abstraction.

We are going to talk you out of building one, most of the time. And when you do build one, we will give you the patterns that hold up in production.

15.1 Before You Build a Multi-Agent System, Don't

A very short list of questions to answer before you even sketch a multi-agent architecture.

Does a single agent fail at this task? If a single well-designed agent can handle the task, adding more agents introduces complexity without value.

Does a workflow (Chapter 11) fail at this task? Many problems that feel like multi-agent are actually workflow problems. A router and specialists is a workflow, and it solves a broad class of “multi-agent” problems cleanly.

Do you have the eval harness to measure a multi-agent system? Multi-agent systems have trajectories across multiple agents, interactions to trace, emergent failure modes to test for. If your evals are not ready for this, your multi-agent system is not either.

Is your team ready to debug one? Multi-agent failures are non-local. A bug in one agent's prompt can manifest as a weird behavior three agents

downstream. Debugging takes real tooling and real patience.

If any of these is a no, stay at workflow or single agent. Come back to multi-agent when the answer changes.

15.2 When Multi-Agent Is Actually the Right Shape

Having been cautious, here are the cases where multi-agent genuinely wins.

Large open-ended research tasks. Problems that need concurrent exploration of multiple lines of inquiry, followed by synthesis. Deep research agents. Multi-source analysis. Competitive intelligence.

Heterogeneous expertise. The task requires genuinely different kinds of thinking at different steps, and co-locating them in one agent produces a confused generalist. Software engineering (where “think like an architect” and “think like a code reviewer” want different prompts) fits here.

Long-running collaborative work. Tasks that run for hours, need independent progress on parallel tracks, and synthesize periodically. Ambitious but rare in production.

Simulation and debate. Any task where the value comes from multiple perspectives interacting. Useful for certain research applications, decision support, creative brainstorming.

Notice what is not here. Customer support. Document review. Compliance checking. Data extraction. The bread and butter of enterprise AI work. These are workflow problems.

Go deeper: Anthropic’s writeup [How we built our multi-agent research system](#) is the most honest public account of the tradeoffs. They explicitly note that the multi-agent approach paid off for research but would be overkill for simpler tasks.

15.3 The Main Multi-Agent Topologies

When you do build one, three topologies cover most of what works.

Orchestrator and workers. A central orchestrator agent plans, delegates subtasks to worker agents, and synthesizes results. The orchestrator is the only agent with the full picture. Workers are scoped to their subtask. Clean boundaries, straightforward evaluation.

Peer-to-peer. Multiple agents, each with a different role, communicate through a shared message bus or state. Useful for simulation, debate, collaborative drafting. Harder to evaluate because the interactions are non-linear.

Hierarchical. Orchestrator directs sub-orchestrators, which direct workers. Necessary for genuinely complex tasks, dangerous for most others because the failure surface grows fast.

Of the three, orchestrator-and-workers is the most common in production and the one we would recommend you default to. Peer-to-peer and hierarchical are more flexible, more expensive, and harder to get right.

15.4 Communication Between Agents

How agents exchange information matters as much as what they do.

Shared state. All agents read from and write to a shared data structure. Scales well, easy to inspect, no message-passing overhead. Most production systems we have seen converge on this.

Message passing. Agents send structured messages to each other. More flexible but introduces ordering issues and harder debugging.

Structured handoffs. An agent finishes, packages a clean summary of what it did, and passes control to the next agent. Used in OpenAI's Agents SDK

handoff pattern.

For most enterprise multi-agent systems, shared state plus structured hand-offs is the simpler architecture and usually the better one.

15.5 Failure Modes You Should Expect

Multi-agent systems fail in specific ways that single-agent systems do not.

Loops. Agent A asks Agent B, Agent B responds asking a clarifying question, Agent A provides the same question it started with, Agent B asks again. Loops can run until a budget stops them. Always set a step budget.

Drift from the goal. Each agent takes a small liberty with the task. Three agents in, the system has quietly redefined what it is doing. Counter this with a canonical goal statement that every agent sees on every turn.

Responsibility fuzziness. Two agents both think they are responsible for something. Or neither does. Tighten roles by giving each agent a scope that is explicitly exclusive of the others.

Compounding errors. Agent A makes a small error. Agent B reasons about A's output as if it were correct. The error compounds. Evals that check each agent's output independently help catch this before it propagates.

Cost explosions. Each agent is an LLM call, often with its own tool calls. Budgets can explode fast. Set cost budgets, alert on approach, kill gracefully on breach.

Knowing these going in lets you design against them. Discovering them in production costs time and money.

Go deeper: Anthropic's [multi-agent research system writeup](#) covers several of these in the "what we learned" section.

15.6 Evaluating Multi-Agent Systems

Evals for multi-agent systems extend the evals for single agents with a few additions.

Per-agent evals. Each agent should have its own eval suite scoped to its role. Router agent gets a routing accuracy eval. Writer agent gets a quality eval.

Trajectory evals. For a given user request, did the system take a reasonable sequence of steps? Did it call the right agents in a sensible order? This usually requires an LLM judge looking at the full trace.

End-to-end evals. Did the final output meet the user's goal? Same as a single-agent eval, but you are testing the full system.

Cost and step evals. How many steps did it take? How much did it cost? Track the distribution and alert on tail cases.

The eval effort for multi-agent systems is roughly 3x the effort for single-agent. Plan for it.

15.7 One Real Example: The Google AI Co-Scientist

One public example worth knowing: Google DeepMind's AI Co-Scientist, a multi-agent system for scientific research assistance.

The system uses six specialized agents (generation, reflection, ranking, evolution, proximity, meta-review) working together to propose research hypotheses. It is a serious production multi-agent system with honest public documentation of its design and evaluation.

We point at it not as a blueprint for enterprise work (your use case is almost certainly not open-ended scientific research) but as a reference for what a well-designed multi-agent system looks like when the task actually requires one.

Go deeper: [Google DeepMind's AI Co-Scientist blog post](#).

15.8 The Honest Short Answer

For 90% of enterprise AI teams reading this chapter, the right multi-agent answer is: not yet. Build workflows. Build single agents where workflows fall short. Keep your evals solid. Ship, learn, iterate.

When a specific task genuinely needs multiple agents working together, you will know. The task will have outgrown what a single agent can handle, your team will have the eval discipline to measure a multi-agent system, and the cost of the additional complexity will be clearly worth it.

Until then, stay simple.

Continue to [Chapter 16: Observability and Tracing](#).

Chapter 16: Observability and Tracing

A system you cannot see is a system you cannot fix. Observability is how you see AI systems in production.

This part of the guide covers the work that turns a shipped system into a system that keeps improving. This chapter is about the foundation of all that work: actually watching what your system is doing, with enough detail to catch problems, answer questions, and feed your evaluation loop.

16.1 What Observability Looks Like for AI Systems

Observability in traditional software is logs, metrics, and traces of what your code did. Observability for AI systems is the same, with a few additions: what prompts were used, what the model returned, what tools were called with what arguments, how many tokens were spent, and what evaluations fired on the output.

Every request through an AI system produces a rich set of data. A complete trace usually includes:

- The user input and any system metadata
- The retrieved context (from any retrieval call)
- The full prompt sent to the model
- The model's raw response

- Any tool calls and their results
- Guardrail decisions (input and output)
- Eval scores (if running in real time)
- Latency for each step
- Token counts and cost
- The final user-facing response

That is a lot of structured data per request. The observability tools of 2026 are built for this shape.

Go deeper: [Arize Phoenix](#) and [LangSmith](#) are the two widely adopted options. Both are free to start.

16.2 Why Traces Matter

A trace is the full record of a single request. The structure is a tree: a root span for the overall request, child spans for each step (retrieval, prompt, tool call, guardrail, model response), grandchild spans for nested work.

Traces matter because AI debugging is fundamentally different from traditional debugging. When a traditional system fails, the stack trace points at the line of code that broke. When an AI system underperforms, there is no line of code that broke. The model gave a bad answer, but why? Was the retrieval wrong? Was the prompt ambiguous? Did the tool return unexpected data?

Without the trace, you cannot answer these questions. With the trace, you can step through the request and see exactly where the quality went off.

This is also why traces are the foundation of production evaluation. The traces become the dataset you eval against. You find a failure, capture the trace, add it as a regression test in your eval suite, and ensure the same bug never comes back.

Go deeper: [OpenTelemetry's GenAI semantic conventions](#) are the emerging open standard for how AI traces should be structured. Useful to know about even if you are using a commercial platform.

16.3 What to Log (and What Not To)

A few guidelines on what a good AI system logs.

Always log: - The system prompt (or a hash of it, with the full version stored separately) - The retrieved context identifiers (not necessarily the full text; you can look those up) - The model name and version - The tool calls and the structured results - Latency and cost per step - Final output - Any guardrail decision that fired

Sometimes log (depending on privacy posture): - The full user input - The full retrieved context text - The full model response

Never log: - Credentials, API keys, or authentication tokens - Sensitive user data unless explicitly required and protected - Raw payloads that might contain PII without redaction

The balance between too little logging (you cannot debug) and too much logging (you have a privacy problem) is a real decision. Make it explicitly, document it, and revisit as your data policies evolve.

16.4 The Metrics That Matter

Beyond individual traces, a few aggregate metrics are worth watching continuously.

Latency distribution. P50, P90, P99 end-to-end latency. P99 catches tail cases that P50 hides.

Cost per request. Total token cost, broken down by model. Tracks whether your system is drifting toward being more expensive over time.

Error rate. Tool failures, model errors, guardrail blocks. A change in error rate is an early warning signal.

Eval scores. If you have evals running in production on a sample, track them over time. This is how you catch quality drift.

Tool call distribution. How often each tool is called. A tool that suddenly starts being called twice as often (or stops being called) is a signal something has changed.

Token usage per request. Prompts getting longer over time is a sign of context bloat. Monitor it.

Most observability platforms surface these automatically. The ones that matter for your product go on the dashboard everyone on the team can see.

16.5 From Traces to Evals and Improvements

The workflow that turns observability into improvement is the loop that matters most:

1. Observe production traffic in traces
2. Spot failures or weak spots (the model said the wrong thing, the retrieval missed, the tool failed)
3. Capture the problem cases as new entries in your eval dataset
4. Fix the system (adjust the prompt, fix the retrieval, change the guardrail)
5. Run the evals to confirm the fix works and nothing else broke
6. Deploy the change
7. Repeat

This is the CC/CD framework in practice. Continuous Calibration is capturing new eval cases from production. Continuous Development is shipping improvements against them.

Teams that do this well ship small changes frequently and their quality trends up over months. Teams that do not do this ship large changes rarely, quality plateaus, and they eventually get passed by competitors who adopted the loop.

16.6 The Minimum Observability Setup

If you are shipping an AI system and have not set up observability yet, the minimum you need before launch:

- A tracing tool capturing every production request (Phoenix, LangSmith, or equivalent)
- A dashboard showing latency, cost, and error rate
- A mechanism for pulling traces into your eval dataset
- A weekly review cadence for looking at failures in production

This can be set up in a day with free tools. The cost of not setting it up is months of flying blind.

Go deeper: [Arize Phoenix Quickstart](#) gets you tracing in under an hour. [LangSmith's observability quickstart](#) is the equivalent for the LangChain ecosystem.

16.7 The Part Nobody Talks About: Culture

A final point that is not technical. Observability tools work when the team actually looks at them. The dashboard that nobody opens is a dashboard that does not exist.

The teams we work with who get the most value from observability build it into their weekly rhythm. A 30-minute review every Monday where the team looks at the week's worst traces, discusses what went wrong, and decides what to add to the eval dataset. No presentations. No deliverables. Just looking at real user traffic together.

Sounds simple. It is the single operational habit that most separates good production AI teams from average ones.

Continue to [Chapter 17: Protocols and Extensibility](#).

Chapter 17: Protocols and Extensibility

The AI world of 2023 was a tower of Babel. Every model provider had its own API shape. Every agent framework had its own tool definitions. Every orchestration library spoke a different dialect of function calling. Building a system that could swap any component for another was a full engineering project.

2026 looks different. Standards have emerged. Protocols have stabilized. The field is moving from bespoke integrations to shared plumbing. This chapter covers the protocols worth knowing about, why they matter for enterprise systems, and how to make your own system extensible.

17.1 Why Protocols Matter for Enterprise

A quick framing. In a startup with five engineers, you can afford to couple your system tightly to whatever model provider you picked. If you ever need to switch, you rewrite the integration. Annoying, but tractable.

In an enterprise with fifty AI systems across departments, that approach is a liability. Each system's integration is different, switching costs are massive, and any new vendor wanting to serve your enterprise has to build a custom integration for every system they touch.

Protocols solve this. A protocol is a shared way of describing how components talk to each other. When your model provider and your tool provider and your data source all speak the same protocol, the integrations become swappable. Switching models becomes a config change. Adding a new data source becomes plug-and-play.

This is exactly why enterprises care about protocols more than startups do. The protocols of 2026 were shaped by enterprise demand.

17.2 Model Context Protocol (MCP)

MCP is the most important protocol you should know about in 2026. Anthropic released it in late 2024, it gained rapid industry adoption in 2025, and by 2026 it is effectively the standard for connecting language models to external tools, data sources, and systems.

The core idea is simple. MCP defines a standard way for a language model (or the application around it) to discover tools, call tools, and receive results. Before MCP, every model provider had its own tool-call format, every agent framework had its own, and integrating a new data source meant writing a custom connector for each.

With MCP, you write one MCP server for your data source or tool. Any MCP-compatible model or framework can now use it. The enterprise implications:

- Data governance becomes cleaner. One MCP server, consistent auth and logging, regardless of which AI system calls it.
- Vendor switching becomes cheap. Swap Claude for GPT without rewriting your tool integrations.
- New AI systems plug into existing data sources without new engineering work.

MCP is also widely adopted in developer tools. Claude Code, Cursor, GitHub Copilot, and most serious IDEs and agent frameworks support MCP as a first-class way of extending their capabilities.

For document and compliance platforms, the implication is meaningful. If a platform exposes its document and compliance data via MCP servers, any AI system (internal or third-party) can access it with a consistent interface. This is the shape enterprise AI integration is heading toward.

Go deeper: [MCP specification and documentation](#) is the primary reference. Anthropic's announcement post [Introducing the Model Context Protocol](#) is the best short introduction.

17.3 Agent Protocols (A2A and Similar)

A newer class of protocols handles agent-to-agent communication. When multiple agents need to coordinate across systems or organizations, you need a shared way for them to exchange goals, updates, and results.

Google announced [A2A \(Agent-to-Agent\)](#) in 2025, with broad industry participation. The protocol defines how agents discover each other, negotiate tasks, and exchange structured updates.

A2A is less mature than MCP, and the adoption curve is earlier. Worth knowing about, probably not worth building your near-term plans around.

For enterprise systems, A2A is most relevant when you expect to connect agents that live in different systems (your system and a partner's system, or two internal systems with different vendors). If every agent lives inside your walls, you can standardize on any internal protocol you like.

17.4 OpenAI Agents SDK

Adjacent to protocols: the Agents SDK from OpenAI is the most widely adopted framework for building agent applications in 2026. It handles the orchestration plumbing (agent loop, tool calls, handoffs, tracing) and includes first-class support for both OpenAI and third-party models.

The SDK matters for this chapter because its handoff pattern is becoming a quasi-protocol for how agents transfer control to each other. You design each agent with a role, specify which other agents it can hand off to, and the SDK manages the transitions.

This is a useful model even if you do not use the SDK directly. It generalizes well to any multi-agent system.

Go deeper: [OpenAI Agents SDK documentation](#). LangGraph's [multi-agent documentation](#) covers the same ideas in a different framework.

17.5 What to Do With All This

A practical framework for the average team reading this chapter.

If you are building a new system today: support MCP as your primary way of integrating tools and data sources. It is stable, it is widely adopted, and the ecosystem is only growing. Write MCP clients when you want your system to consume external tools. Write MCP servers when you want your system to be consumed by other AI tools.

If you have an existing system: assess where protocol adoption reduces your integration work. The first place is usually tool integrations. MCP-ifying your internal tools is a bounded project with clear ROI.

If you are a PM or designer thinking about a roadmap: protocols let you unbundle the roadmap decision from the vendor decision. You can commit to

a capability (integrate with the compliance database, extend with document lookup) independently of which model or framework you will use.

If you are thinking about multi-agent work: watch A2A and related protocols, but do not bet your architecture on them yet. They will mature; your production systems cannot wait for them to.

17.6 Making Your Own System Extensible

Protocols are one kind of extensibility. The other kind is internal: designing your own AI system so new capabilities can be added without rewriting the core.

A few principles that show up in systems that age well.

Tool contracts are frozen. Once a tool's interface is defined, you do not change its input or output shape. Add new tools rather than changing existing ones.

Agent roles are named and isolated. Each agent has a clearly documented role and boundary. Adding a new agent is a clean operation. Changing an existing agent's role is rare.

Prompts are versioned. System prompts get treated like code. Git history, change logs, rollback capability.

Evals cover the seams. When new capabilities are added, the eval suite tests the seams (agent handoffs, tool boundaries, retrieval edges). This catches the failures that happen at component boundaries.

Configuration is external. Model choice, prompt versions, retrieval parameters, guardrail thresholds. All in config, not hard-coded. So they can change independently.

These are not protocol standards. They are internal discipline. Done well, they make your system feel extensible even without external protocols. Done

poorly, no amount of protocol adoption will save you.

17.7 The Horizon for Protocols

The 2026 state of protocols is a snapshot. Things are still moving.

MCP is settled for tool connection. A2A and friends are converging for agent communication. New protocols are being proposed for persistent memory sharing, for cross-system evaluation, and for safety attestation. Some will stabilize and get adopted. Most will not.

The useful posture is to follow the space lightly, adopt standards once they are proven, and not over-commit to any protocol that is less than a year old. Enterprise is the patient customer. That is a strategic advantage here.

Continue to [Chapter 18: Production Readiness Checklist](#).

Chapter 18: Production Readiness Checklist

Most AI systems are ready for production long before the team believes they are. A few are shipped long before they should be. The difference between the two is rarely instinct. It is whether the team has run through a checklist, honestly.

This chapter is that checklist. It is the one we use at LevelUp Labs when we advise on launch decisions for client AI systems. It is also the rubric we use to triage systems that are in production and underperforming: more often than you would expect, the answer traces to a checklist item that was quietly skipped.

Keep this chapter handy. Run through it before any AI system you ship. Run through it again quarterly on any AI system you operate.

18.1 The Core Principle

A production AI system is not ready when the happy path works. It is ready when the unhappy paths are handled gracefully.

Every other item on this checklist is an instance of that principle. The question to hold in your head as you read the rest of this chapter: when the thing goes wrong, what happens?

18.2 Pre-Launch Checklist

These are the items worth confirming before the first real user sees the system.

Problem and Scope

- ☐ The problem statement has a named user and a measurable outcome
- ☐ The scope is narrow enough that a single eval suite can cover it
- ☐ A human baseline exists for the task (you know how long it takes a human, how often they get it wrong)
- ☐ The success criteria for launch are explicit and written down

Evals

- ☐ A code-based eval suite covers the main patterns with at least 30 cases
- ☐ An LLM-as-judge eval is in place for any subjective quality dimension (tone, helpfulness, groundedness)
- ☐ The LLM judge has been calibrated against human labels, with agreement above 80%
- ☐ Eval scores are above the launch bar
- ☐ Eval suite runs on every code change (in CI or equivalent)

Guardrails

- ☐ Input sanitization is in place
- ☐ PII redaction runs on input if the system sees sensitive data
- ☐ Prompt injection detection is active if external users can send input
- ☐ Output groundedness check runs for any retrieval-based system
- ☐ Action boundary enforcement runs for any system with action-taking tools

- ☐ A graceful fallback path exists for when guardrails fire

Observability

- ☐ Every production request is traced end to end
- ☐ Latency, cost, and error rate are visible on a dashboard the team checks weekly
- ☐ A mechanism exists for promoting production failures into the eval dataset
- ☐ Alerts fire on cost spikes, error rate increases, or latency tail cases

Safety

- ☐ A rollback path is tested and ready (feature flag, version pin, or equivalent)
- ☐ Incident response ownership is assigned (specific person, specific pager rotation)
- ☐ Rate limits are in place to prevent runaway cost in the case of a bug or abuse
- ☐ Human review is available as a fallback for the cases the system refuses or is uncertain about

User Experience

- ☐ The system gracefully handles its own failures (model error, tool timeout, retrieval empty)
- ☐ Loading states, error states, and empty states are designed
- ☐ Users have a clear path to escalate when the AI is wrong
- ☐ Confidence or uncertainty is communicated where relevant

Documentation

- ☐ The system's scope is documented (what it does, what it does not)
- ☐ The prompt, the model, and the retrieval strategy are versioned and documented
- ☐ The eval suite is documented with the rubric for each eval
- ☐ An operational runbook exists for common incidents

If every item is checked, you are ready to launch. If not, the unchecked items are a live list of things to handle before the launch or to acknowledge as known risks if you proceed anyway.

18.3 Post-Launch Checklist

Production readiness is not just about launch day. It is about the operational practice that keeps the system healthy over months.

Weekly

- ☐ Review the week's production traces. Pick the 10 worst cases. Understand what went wrong.
- ☐ Check the dashboard. Any metric that changed by more than 20% warrants an investigation.
- ☐ Promote any real failures found in production into the eval dataset as regression tests.

Monthly

- ☐ Run the full eval suite from scratch. Compare scores to last month. Investigate any drops.

- ☐ Review the guardrail logs. Are the guardrails firing on the cases you expected? Are new kinds of bad input appearing?
- ☐ Survey or interview three to five real users. Where is the system working? Where is it falling short?

Quarterly

- ☐ Recalibrate the LLM judges against human labels. Model drift and prompt edits both erode calibration over time.
- ☐ Review the eval dataset itself. Are the cases still representative of real traffic? Add new ones, retire outdated ones.
- ☐ Consider whether a model upgrade (to a newer or different model) would lift quality on the current eval set.
- ☐ Review cost per request. Is the system more expensive than it needs to be? Can cheaper models handle some components?

Annually

- ☐ Revisit the problem statement. Is the system still solving the right problem? Has the user's workflow evolved?
- ☐ Audit the full guardrail and observability stack. Are there gaps that did not exist when you launched?
- ☐ Do a full incident retrospective. Which incidents happened this year? What systemic changes prevent them?

Teams that run these cadences consistently are the ones whose AI systems get better over time. Teams that do not drift quietly.

18.4 The Signals You Are Drifting

A few early-warning signs that an AI system is drifting from its production readiness state.

- Eval scores are not being looked at weekly
- The eval dataset has not been updated in two months or more
- The team disagrees on whether a recent output was good
- Cost per request is creeping up without an explanation
- User complaints are being responded to individually rather than traced to systemic issues
- New features are being added faster than new evals
- Guardrail logs are not being reviewed

If three or more of these are true, you are drifting. Recoverable, but worth a deliberate re-investment in the production practice.

18.5 A Word on Shipping Imperfect Systems

One last point. The checklist above is a bar to clear when you can, and a deliberate tradeoff when you cannot. Real systems sometimes ship with items unchecked because the business reason outweighs the risk.

When that happens, the right move is to acknowledge the risk explicitly, put a deadline on closing the gap, and make the gap visible. A system that ships with known risks and a plan is safer than a system that ships pretending everything is handled.

The teams we worry about are not the ones that ship with known gaps. They are the ones that ship thinking they are ready when they have never run through a checklist. The checklist is the defense against that.

Continue to [Chapter 19: Role-Based Learning Tracks](#).

Chapter 19: Role-Based Learning Tracks

You do not need to read this guide cover to cover to get value from it. This chapter gives you three curated reading paths, one for each of the three roles we see most often on enterprise product teams: Product Managers, UX Designers, and Engineers.

Each track recommends chapters in a specific order, with a short note on why each chapter matters for that role. Each track is designed to take roughly 30 to 45 days at a pace of two to three chapters per week.

19.1 Product Manager Track

Your job is to decide what to build, in what order, with what scope. The chapters that matter most for you are the ones that give you a taste for what is feasible, what is expensive, and what separates AI products that ship from ones that do not.

Week 1: Language and Landscape - Chapter 1: The Language of Generative and Agentic AI. You need the vocabulary to talk to your engineers without them having to translate - Chapter 2: What Enterprises Are Building. Shapes your intuition for what is realistic in 2026

Week 2: Designing the Product - Chapter 4: Problem-First Design, Revisited. The chapter most directly relevant to your daily work - Chapter 3: Mod-

els: How to Choose. Decide the rough cost and capability envelope before you commit to a roadmap

Week 3: The Evaluation Muscle - Chapter 6: Why Evals Are the Real Work. Sets the right expectations with your engineering team - Chapter 8: LLM-as-Judge and Calibration. Understand how subjective quality gets measured (important for scoping timeline)

Week 4: Shape of the System - Chapter 10: From Single Calls to Agents. Know when to push back on “let’s build an agent” - Chapter 11: Workflow and Router Patterns. The shape most of your roadmap should default to

Week 5: Ship and Operate - Chapter 18: Production Readiness Checklist. Use this as your launch criteria - Chapter 2 again. Now reread with production experience and see what lands differently

Skip on first pass: Chapters 7 (code-based evals), 12 (tool use), 13 (retrieval), 14 (memory), 15 (multi-agent), 16 (observability), 17 (protocols). These are where your engineers live. Read them when a specific decision comes up.

Revisit when: You are scoping a new AI project. You are pushing back on a complex architecture proposal. You are trying to decide between two vendors.

19.2 UX Designer Track

Your job is to design the interaction between the human and the AI system. The chapters that matter most for you are the ones that ground you in what the system can do, how it fails, and how to design for uncertainty and trust.

Week 1: Language and What the System Actually Is - Chapter 1: The Language of Generative and Agentic AI. Shared vocabulary with the rest of

the team - Chapter 5: Prompting and Context Engineering. The designable surface you might not have realized you could influence

Week 2: Designing for What AI Systems Get Right and Wrong - Chapter 4: Problem-First Design, Revisited. Start with the user, always - Chapter 9: Guardrails: Input and Output. The safety layer that shapes how the system refuses, redirects, and communicates uncertainty

Week 3: The Experience of Trust - Chapter 8: LLM-as-Judge and Calibration. How subjective quality gets measured; think about how this maps to user perception - Chapter 11: Workflow and Router Patterns. The chapter that most shapes how users experience the “doors” in an AI product

Week 4: When the System Acts - Chapter 12: Tool Use and Actions. If your product has actions, this is the chapter on the experience of confirmation, drafting, and undoing - Chapter 13: Retrieval. Understand the shape of systems that ground their answers in documents; relevant to compliance, research, and document-heavy UX

Week 5: Production Reality - Chapter 18: Production Readiness Checklist. The UX items on this list are your direct responsibility - Chapter 6: Why Evals Are the Real Work. Gives you language for why the system’s behavior changes over time

Skip on first pass: Chapters 3 (model selection), 7 (code-based evals), 10 (agent spectrum), 14 (memory), 15 (multi-agent), 16 (observability), 17 (protocols). Come back when you have a specific design question that touches them.

Revisit when: You are designing a confirmation flow. You are designing an empty state or error state. You are trying to communicate system confidence to a user. You are designing for a domain (compliance, regulated content) where the stakes of AI failure are high.

19.3 Engineer Track

Your job is to build the thing. You need everything, in roughly the order it is presented. A faster pass through the chapters you already know, deeper engagement on the ones you do not.

Week 1: Ground Yourself - Chapter 1: The Language of Generative and Agentic AI. Quick pass, catch any unfamiliar terms - Chapter 3: Models: How to Choose. The decision framework for your model choice - Chapter 5: Prompting and Context Engineering. Probably your most-used chapter

Week 2: Build the Eval Discipline First - Chapter 6: Why Evals Are the Real Work. The argument for why this is the most important chapter for you - Chapter 7: Code-Based Evals. Build these first, always - Chapter 8: LLM-as-Judge and Calibration. Add these when you need them

Week 3: Build the Safety Layer - Chapter 9: Guardrails: Input and Output. Input sanitization day one, rest as you discover need - Chapter 12: Tool Use and Actions. Always start read-only, always scope permissions - Chapter 13: Retrieval. For any system with documents

Week 4: Shape the System - Chapter 10: From Single Calls to Agents. Stay at the simplest level that works - Chapter 11: Workflow and Router Patterns. Default architecture for most enterprise systems - Chapter 14: Memory and Long-Running Agents. Only if your use case needs it

Week 5: Ship It - Chapter 16: Observability and Tracing. Set this up before launch, not after - Chapter 17: Protocols and Extensibility. MCP is the standard worth learning - Chapter 18: Production Readiness Checklist. Run through this before every launch

Only when the need arises: Chapter 15 (multi-agent systems). Most engineers will not build one. Those who do need the full chapter.

Revisit when: You are choosing a new architecture. You are debugging a weird production failure. You are trying to decide whether to build or to rely

on a protocol. Quarterly, reread Chapter 6 and the Production Readiness Checklist.

19.4 Shared Homework Across All Tracks

Regardless of role, three things are worth doing alongside the reading.

The companion evals course. [AI Evals for Everyone](#) goes deeper on the mechanics with video lessons and live sessions. Anyone on your team who wants the full treatment can work through it alongside Part 3 of this guide.

Subscribe to LevelUp Labs' monthly paper roundup. Hosted in the [awesome-generative-ai-guide repo](#), updated monthly with the most important research of the month, summarized for practitioners.

Teach what you read. Pick a topic from this guide, find a colleague who does not know it, and explain it in 15 minutes. If you cannot explain it, you do not know it yet.

19.5 For Leaders Reading This

A bonus fourth track, for engineering managers, VPs, and heads of product.

Your most valuable reads are: - Chapter 2 (what enterprises are building) for roadmap intuition - Chapter 6 (why evals are the real work) to understand what you are staffing for - Chapter 10 (agent spectrum) to push back on over-ambitious architecture proposals - Chapter 18 (production readiness) as a launch review rubric

Fewer chapters, more revisits. The practice for leaders is making sure these are the lens through which AI decisions get filtered across the team.

Continue to [Chapter 20: The Horizon](#).

Chapter 20: The Horizon

A book about AI in 2026 that tries to predict 2027 is probably going to look embarrassing in 2027. We will try anyway, but carefully.

This final chapter is about the themes worth watching over the next one to two years. Some of them are already shaping production systems. Some are still research with strong commercial signal. All of them are likely to change the advice in this guide, somewhere, in the direction everything is heading.

20.1 Reasoning Keeps Getting Better and Cheaper

The most consequential trend of 2025 and 2026 is that reasoning models have become both more capable and more affordable. OpenAI's o-series, Anthropic's Claude with extended thinking, Google's Gemini with thinking, and DeepSeek's open reasoning models have all pushed the ceiling of what AI systems can do on multi-step problems.

The direction is clear: the next two years bring reasoning models that are cheaper per token, faster per response, and more reliable on harder problems. The implications for how you design systems:

- Tasks that used to need a multi-step workflow will increasingly be solvable with a single reasoning-model call
- Agent loops become more reliable as the planner at their core gets better

- The cost calculation for reasoning models shifts in their favor; by late 2026, reasoning is the default for non-trivial tasks

If you are designing a system today, plan for reasoning to be more central in your architecture a year from now. Build interfaces that can absorb a reasoning-model upgrade without requiring rewrites.

20.2 Long Context, Really

Models with 1M-token context windows are widely available in 2026. In 2027, we expect larger windows with meaningfully better attention quality across the full range (the “lost in the middle” problem softens as models get better at long context).

The practical shift: retrieval becomes less necessary for some use cases, because you can just put the whole document in the prompt. This does not kill retrieval (cost and efficiency still matter), but it changes the design decisions. For a single-user, single-document interaction where cost is not the constraint, just putting the document in context is becoming a viable pattern.

For multi-user, multi-document, or high-volume systems, retrieval remains the right answer. But the line between “this needs retrieval” and “this can just fit in context” is moving.

20.3 Multimodal by Default

Text-only models feel increasingly limited. Vision is table stakes in 2026. Audio and video inputs are becoming standard. The next year brings native multimodal models that blur the lines between input modalities.

For document-heavy use cases, the relevant shift is that mixed-media documents (PDFs with tables, charts, forms, images, text) can be processed end-to-end by a single model without specialized pipelines. The OCR-then-extract architecture of 2023 is being replaced by vision-language models that handle everything at once.

Expect significant UX shifts as well. Designing for a system that can look at a screenshot, read a form, and answer a question about it is different from designing for a text-only system.

20.4 Open Models Close More of the Gap

The open-model ecosystem has become genuinely competitive in 2026. Llama, Qwen, DeepSeek, and Mistral all ship models that meet or exceed closed-model performance on many benchmarks.

Two forces keep this going. Infrastructure for hosting open models has gotten cheaper (Together, Fireworks, Groq, Cerebras). Post-training techniques have gotten better at extracting strong performance from smaller parameter counts.

The likely state in 2027: frontier capability remains closed (reasoning, multi-modal, edge cases), but a large share of enterprise workloads can be served by open models at meaningfully lower cost. Teams that structure their systems to be model-agnostic will have the most flexibility as this plays out.

20.5 Autonomy, Carefully

Autonomous agents (systems that take action without real-time human approval) have been promised for years. The 2026 state is still that they work in narrow domains with careful guardrails, and they fail unpredictably outside those domains.

The trajectory is not a sudden breakthrough into full autonomy. It is a steady expansion of the domains where autonomous action is safe, driven by better models, better evals, and better guardrails.

The practical implication for enterprise systems: expect to automate more steps than you do today, but expect that automation to come through a slow widening of “what the system is allowed to do autonomously” rather than a leap to full autonomy. The progression in Chapter 12 (read-only → draft-then-confirm → action with guardrails → autonomous) is the shape of this change.

20.6 Regulation and Governance

Regulatory and governance pressure on AI systems is increasing globally. The EU AI Act is in effect. Several US states have passed AI-specific legislation. Industry-specific rules (financial services, healthcare, legal) are adding AI-specific compliance requirements.

For enterprise teams, this means that the “production readiness” bar is rising. Audit trails, documented decision processes, explainability, and bias testing are moving from nice-to-have to required. The checklist in Chapter 18 is conservative by today’s standards; it will be table stakes by 2027.

The silver lining is that much of this overlaps with what good production practice already demands. Eval suites, observability, guardrails, and human-in-the-loop patterns are not just good engineering; they are also the scaffolding for regulatory compliance. The teams that invested early in the practice we describe in Part 3 and Part 5 of this guide will have a much easier path through the regulatory landscape.

Go deeper: [EU AI Act summary](#) for the legal shape. [NIST AI Risk Management Framework](#) for a widely-cited US reference.

20.7 The Enterprise AI Platform

Looking further out, a meta-trend worth naming: enterprise AI is slowly turning into a platform discipline. Individual AI systems at individual teams are giving way to shared AI platforms that serve the whole organization.

The signs are everywhere. Centralized eval teams. Shared guardrail libraries. Company-wide observability for AI. Internal MCP servers for all corporate data sources. Central model governance. Uniform incident response for AI failures.

If this is the direction, the implication for a product team is that your AI system will increasingly be built on top of shared organizational infrastructure, not as a standalone project. The skills you build over the next year, the ones this guide is trying to transfer, will eventually be the baseline for working inside that platform.

20.8 What Does Not Change

A lot does not change. The durable parts of the practice:

- Problem-first design still matters more than any specific technology choice
- Evaluation is still the core muscle for shipping AI that actually works
- Workflows still beat agents for most enterprise problems
- Tool design still matters more than tool count
- Retrieval quality still depends on chunking more than on the vector database
- Observability is still the foundation of continuous improvement
- The teams that get good at AI are the ones that invest in a durable practice around evaluation, observability, and iteration

These will still be true in 2027. They will still be true in 2030. The models will change. The practice is durable.

20.9 What to Do With This Chapter

Do not plan against specific predictions. We could be wrong about reasoning adoption curves, about open-model parity, about regulatory pace.

Do plan against the direction. Reasoning is getting more central. Multimodal is becoming the default. Open models are closing the gap. Governance is rising. Enterprise AI is turning into a platform.

Design your systems so that the inevitable changes are absorbable. Protocols, model-agnostic architecture, strong evals, clean tool contracts, external configuration. Do this and the next two years of field change are a tailwind rather than a threat.

20.10 Final Word

You finished the guide.

The reward for finishing a guide like this is not the finish. It is the next system you build. The work is in the building, in the evals, in the traces at 11pm, in the quarterly recalibration, in the weekly habit of looking at real user traffic together.

We hope this guide gives you the scaffolding to do that work faster and better. The builds are yours.

Keep us posted on what you ship.

LevelUp Labs

Continue to the [Master Resource Index](#), or return to the [Table of Contents](#).

Master Resource Index

Every external resource referenced across the twenty chapters, organized three ways: by source, by topic, and by type of resource.

Use this as your starting point when you want to go deeper on a specific topic or when you are trying to find that one link you remember reading about in Chapter 11.

By Source

LevelUp Labs

The most important single source for this guide. Free, actively maintained, built for practitioners.

- [awesome-generative-ai-guide](#): the main repo. Paper roundups, free courses, interview prep.
- [AI Evals for Everyone](#): LevelUp Labs' companion course on evaluation. Threaded throughout Part 3 of this guide.
- [Generative AI Genius](#): LevelUp Labs' free course on building generative AI applications end to end.
- [Applied LLMs Mastery 2024](#): 10-week course on LLM applications, foundations through deployment.

- [Agents 101 guide](#)
- [3-day RAG roadmap](#)
- [5-day LLM foundations roadmap](#)
- [5-day LLM agents roadmap](#)
- [Fine-Tuning 101 guide](#)
- [Introduction to Multimodal LLMs](#)
- [Securing Agentic AI Systems](#)

Anthropic

The clearest applied writing in the field from a model provider.

- [Building Effective Agents](#): referenced repeatedly. Read end to end.
- [Effective Context Engineering for AI Agents](#)
- [How we built our multi-agent research system](#)
- [Mapping the Mind of a Large Language Model](#)
- [Introducing the Model Context Protocol](#)
- [Contextual Retrieval](#)
- [Prompt Engineering Overview](#)
- [Tool Use documentation](#)
- [Extended Thinking documentation](#)
- [Structured Outputs documentation](#)
- [Citations documentation](#)
- [Memory Tool documentation](#)
- [Economic Index](#)

OpenAI

- [Learning to Reason with LLMs](#): OpenAI's post on o-series reasoning
- [Why language models hallucinate](#)
- [Prompt Engineering guide](#)

- [Function Calling guide](#)
- [Structured Outputs guide](#)
- [Reasoning Guide](#)
- [Embeddings guide](#)
- [Evals guide](#)
- [Tokenizer tool](#)
- [Agents SDK documentation](#)
- [OpenAI customer stories](#)

Google and Google DeepMind

- [Accelerating scientific breakthroughs with an AI co-scientist](#)
- [Gemini models documentation](#)
- [Vertex AI Grounding documentation](#)

Meta

- [Llama research](#)
- [Llama Guard research](#)

DeepLearning.AI

Andrew Ng's free learning platform. The cleanest, most accessible technical education in AI.

- [Generative AI for Everyone](#): best non-technical intro
- [DeepLearning.AI Short Courses](#): 60+ free short courses on specific AI topics, many co-taught with Anthropic, OpenAI, and other providers

Observability Platforms

- [Arize Phoenix](#): widely adopted observability platform

- [Arize Phoenix Quickstart](#)
- [LangSmith](#): LangChain's observability platform
- [LangSmith observability quickstart](#)
- [Promptfoo](#): open-source eval framework
- [OpenTelemetry GenAI semantic conventions](#): the emerging open standard

Retrieval and Vector Databases

- [Pinecone Hybrid Search](#)
- [Weaviate Hybrid Search](#)
- [pgvector documentation](#)
- [Cohere Rerank](#)
- [Pinecone on Rerankers](#)
- [Microsoft GraphRAG](#)
- [LlamaIndex advanced retrieval](#)

Frameworks and Agent Tooling

- [LangGraph documentation](#)
- [LangGraph human-in-the-loop](#)
- [LangGraph memory](#)
- [LangGraph multi-agent](#)
- [LangGraph persistence](#)
- [LangChain context engineering](#)
- [Letta](#): open-source memory system (formerly MemGPT)

Guardrail Tools

- [NVIDIA NeMo Guardrails](#)
- [Microsoft Presidio](#): PII detection

- [Lakera Guard](#): prompt injection and jailbreak detection
- [Prompt Armor](#)

Protocols

- [Model Context Protocol \(MCP\) spec](#)
- [A2A \(Agent-to-Agent\)](#)

Enterprise AI Reports

- [McKinsey State of AI](#)
- [Menlo Ventures State of AI in the Enterprise](#)
- [a16z Enterprise AI report](#)

Regulation and Governance

- [EU AI Act](#)
- [NIST AI Risk Management Framework](#)

Product and Design References

- [Marty Cagan's Product Discovery](#)
- [Shreyas Doshi's writing](#)

Benchmarks and Model Comparison

- [HELM](#): Stanford's benchmarking effort
- [LMArena](#): crowdsourced head-to-head model comparisons
- [Hugging Face Open LLM Leaderboard](#)

By Topic

Foundations and Vocabulary

- LevelUp Labs' [awesome-generative-ai-guide](#)
- DeepLearning.AI's [Generative AI for Everyone](#)
- LevelUp Labs' [Generative AI Genius](#)

Enterprise Landscape

- [McKinsey State of AI](#)
- [Menlo Ventures](#)
- [a16z Enterprise AI](#)
- [Anthropic Economic Index](#)
- [OpenAI customer stories](#)

Model Selection

- [HELM](#)
- [LMArena](#)
- [Hugging Face Open LLM Leaderboard](#)
- Provider model pages (Anthropic, OpenAI, Google, Meta)

Problem-First Design

- [Marty Cagan's Product Discovery](#)
- [Building Effective Agents](#)

Prompting and Context Engineering

- [Anthropic Prompt Engineering](#)

- [OpenAI Prompt Engineering](#)
- [Anthropic Context Engineering](#)
- [LangChain Context Engineering](#)
- LevelUp Labs' [Applied LLMs Mastery Week 2](#)

Evaluation

- **LevelUp's [AI Evals for Everyone](#)**: the headline recommendation
- [Arize Phoenix evals](#)
- [LangSmith evaluation](#)
- [Promptfoo](#)
- [OpenAI Evals](#)
- [Arize on LLM-as-Judge](#)

Guardrails

- [NeMo Guardrails](#)
- [Presidio](#)
- [Lakera Guard](#)
- [Llama Guard](#)

Agents and Workflows

- [Building Effective Agents](#): the single best reference
- LevelUp Labs' [Agents 101 guide](#)
- [OpenAI Agents SDK](#)
- [LangGraph documentation](#)

Tool Use

- [Anthropic Tool Use](#)

- [OpenAI Function Calling](#)
- [Arize on Tool Use](#)

Retrieval

- [LevelUp Labs' 3-day RAG roadmap](#)
- [Anthropic Contextual Retrieval](#)
- [OpenAI Embeddings](#)
- [Pinecone Hybrid Search](#)
- [Microsoft GraphRAG](#)

Memory

- [Anthropic Memory Tool](#)
- [LangGraph Memory](#)
- [Letta](#)

Multi-Agent

- [Anthropic multi-agent research system](#)
- [Google AI Co-Scientist](#)
- [LangGraph multi-agent](#)

Observability

- [Arize Phoenix](#)
- [LangSmith](#)
- [OpenTelemetry GenAI](#)

Protocols

- [MCP](#)
- [Anthropic MCP announcement](#)
- [A2A](#)
- [OpenAI Agents SDK](#)

Regulation and Governance

- [EU AI Act](#)
 - [NIST AI Risk Management Framework](#)
-

By Type

Free Courses

- [AI Evals for Everyone](#): **LevelUp**, strongly recommended
- [Generative AI Genius](#): **LevelUp**
- [Applied LLMs Mastery 2024](#): **LevelUp**
- [Generative AI for Everyone](#): DeepLearning.AI, non-technical
- [DeepLearning.AI Short Courses](#): 60+ courses on specific topics

Provider Documentation Starting Points

- [Anthropic Docs](#)
- [OpenAI Platform Docs](#)
- [Google AI Docs](#)
- [Meta Llama](#)

Industry Reports (Annual or Ongoing)

- McKinsey State of AI
- Menlo Ventures State of AI in the Enterprise
- a16z Enterprise AI
- Anthropic Economic Index

Newsletters and Ongoing Sources

- LevelUp Labs' monthly paper roundup in [awesome-generative-ai-guide](#)
- Anthropic Engineering Blog (anthropic.com/engineering)
- OpenAI Research (openai.com/research, openai.com/index)
- LangChain Blog (blog.langchain.com)

Return to the [Table of Contents](#).